SymCorr: A Compiler Framework for Automated Construction and Efficient Evaluation of Lattice QCD Correlation Functions

Tanishk Shrimal[a,*], M. Padmanath[a]

[a]The Institute of Mathematical Sciences Chennai, IV Cross Road, CIT Campus, Taramani, Chennai, 600113, Tamil Nadu, India

ABSTRACT

a

1. Introduction

The strong coupling of Quantum ChromoDynamics (QCD) at low energies precludes the use of perturbative approaches, where hadrons emerge as the relevant effective degrees of freedom. Consequently, hadron physics investigations in the low energy regime rely on nonperturbative approaches such as those using the lattice QCD framework. Generation of lattice QCD ensembles through Monte-Carlo simulations and estimation of fermion matrix determinant as well as inversions of the fermion matrix consume substantial fraction of the computer time in lattice QCD calculation and have been extensively discussed and investigated since the inception of lattice QCD []. Physical observables in hadron-physics investigations are extracted from Euclidean correlation functions. In the early years of lattice QCD, these calculations primarily targeted ground-state hadron properties and were comparatively inexpensive. The discovery and study of numerous heavy and excited hadrons over the past several decades, however, necessitated increasingly sophisticated spectroscopy calculations, notably those employing Euclidean correlation matrices and variational analyses to determine the finite-volume spectrum and its properties. As the size and complexity of these correlation matrices increase, the associated operator construction, Wick contractions, and numerical evaluation become increasingly demanding, resulting in substantial rise in computational costs.

Modern day hadron spectroscopy proceeds through evaluation of two point Euclidean correlation matrices

$$C_{ij}(t-t_0) = \left\langle \mathcal{O}_i(t)\, \mathcal{O}_j^{\dagger}(t_0) \right\rangle \tag{1}$$

for a basis of interpolators/operators $\{O_i\}$ that carry the quantum numbers of the desired hadronic state, where t and t_0 refers to the sink and the source timeslices. In this context, the new challenges include a systematic construction of interpolating operators that enter the evaluation of correlation matrices, handling the allowed Wick contractions for every combination of interpolating operators, keeping track of the fermionic signs with permutations, flavor, spin, color and spatial indices faithfully, etc. The amount of bookkeeping therefore grows as one moves from meson operators to baryons, multi-hadron operators and nuclei. Additional conversions and reductions need to be made to express the correlation matrices in terms of the objects defining operator structures as well as the fermion matrix inverses. Note that different matrix elements carry similarities and share quark-propagation data, operator-dependent building blocks, and often portions of the resulting contraction expressions. This observation suggests opportunities to identify repeated (sub)objects IO and computations, reducing the computational redundancy. While a systematic road-map has already been laid out for such excited hadron spectroscopic investigations in the literature, performing them manually for every new channel being studied requires substantial implementation effort and can introduce bookkeeping errors.

There already exists tools to handle the operator construction and correlation matrix evaluation. Tools such as Multi-Hadron Interpolators (MHI) [1] and `OpTion` [2] provides a systematic construction of multi-hadron lattice operators projected to finite-volume irreps, total lab frame momentum, spin, and internal quantum numbers. The QCT package [3] generates `C++` codes compatible with `QDP++` [4] to perform Wick contractions for a given set of hadronic currents/operators expressed in terms of (anti)quark fields, without further optimization for computational efficiency. The Redstar framework [5] expresses each element in correlation matrices as a sum of graphs (Wick contractions) and utilizes uses graph-contraction algorithms to reduce redundant evaluations and to organize optimized correlator evaluations in a distillation-based calculational setup [6] using tensor contractions for execution on CPUs and GPUs. Another recent algorithmic work explored determinant representations of Wick contractions and methods for identifying smaller sets of determinant terms that are sufficient for evaluating nuclear correlation functions [7].

The emphasis of the SymCorr package is on having a complete automated transformation from a symbolic quark-level operator description to a numerical calculation within the distillation prescription [6] as a sequence of explicit compilation stages. Once the user provides the symbolic definition of a basis of operators in terms of quark and gauge fields, the operator representations, Wick contractions, automated translation to backend-independent physics

*Corresponding author

tanishks@imsc.res.in (T. Shrimal); padmanath@imsc.res.in (M. Padmanath)

ORCID(s): 0000-0001-0000-0000 (T. Shrimal); 0000-0001-0000-0000 (M. Padmanath)

objects, distillation lowering, execution planning, and numerical evaluation are organized as distinct layers of the software. This staged organization reduces manual intervention in correlator-specific algebra and implementation, while maintaining a clear separation between the physical definition of an operator and the details of its numerical evaluation. It also makes the intermediate algebraic transformations explicit and amenable to inspection and validation, while allowing numerical execution strategies to evolve and adapt independent of the underlying physics description of the interpolating operators.

2. Overview and scope of SymCorr

SymCorr package facilitates the computation of Euclidean correlation matrices in the distillation framework, serving itself as a generalized automated platform in computation of arbitrary multi-hadron correlation matrices in finite-volume, with the assumption that the relevant lattice numerical data and the operator basis details are provided by the user. We begin this section by briefly discussing numerical objects relevant for the distillation framework for completeness. In the conventional non-distillation setup, the measurements involve computation of correlation matrices given in Eq. 1 for a given operator basis and varying values of t and t_0. These involve appropriate tensor index contractions between spatial-color-spin indices of the hadron interpolators with those in the quark line objects referred to as the quark propagators.

The computational cost of such tensor index contractions spanning through the space-color-spin index values ($\mathcal{O}(10^6)$ for a 40^3 spatial lattice) can become prohibitively expensive when then system being studied is large or complex such as multi-hadron operators, tetraquarks, pentaquarks, nuclei, etc. In the distillation framework, the combined space-color index is replaced by a single index, which could run over a appropriately chosen eigenmode subset of discretized gauge-covariant Laplacian $\{v^{(i)}(t)\}$. Such a replacement can be facilitated through quark smearing program defined in terms of $\{v^{(i)}(t)\}$ as

$$\begin{aligned} &\tilde{q}(t) = \square(t)q(t), \qquad \tilde{\bar{q}}(t) = \bar{q}(t)\square(t), \\ &\text{where} \quad \square(t) = V(t)V^\dagger(t) = \sum_{i=1}^{N_v} |v^{(i)}><v^{(i)}|(t), \\ &\text{and} \quad V(t) = \left(v^{(1)}(t), v^{(2)}(t), \ldots, v^{(N_v)}(t)\right). \end{aligned} \tag{2}$$

The $\square(t)$ operator acts as a low pass filter or a projection operator, with the lowest N_v modes. Inserting such projection operators for the quark fields in the conventional correlations, followed regrouping the objects to form new objects such as

$$\tau_f(t,t_0) = V^\dagger(t) M_f^{-1}(t,t_0) V(t_0) \tag{3}$$

$$\Phi_{\mathcal{O}}(t) = V^\dagger(t)\, \mathcal{O}\, V(t). \tag{4}$$

referred to as the perambulators and elementals/kernels, which takes over the information from the quark propagator $M_f^{-1}(t,t_0)$ and hadronic operator $\Phi_A(t)$, respectively. The objects $\tau_f(t,t_0)$ and $\Phi_{\mathcal{O}}$ carry indices running of the $N_v \sim 100 - 1000$ modes considered facilitating contractions of tensor indices with reduced computational cost. This novel procedure was not only found to reduce the computational cost requirement involved in tensor contractions, but also found to provide access to all-to-all quark propagators, to source momentum projection in hadron operator, multi-hadron operators, etc. The expensive quark-propagation information can be prepared independently of a particular spectroscopy operator basis, while changes to momenta, spin structures, or spatial constructions modify the operator-dependent factors. Within the distillation framework, the correlation matrix elements can be written in a compact form as

$$C_{ij}(t,t_0) = \sum_\alpha c_\alpha \ \mathrm{Contr}_\alpha\left[\{\Phi\},\{\tau\}\right], \tag{5}$$

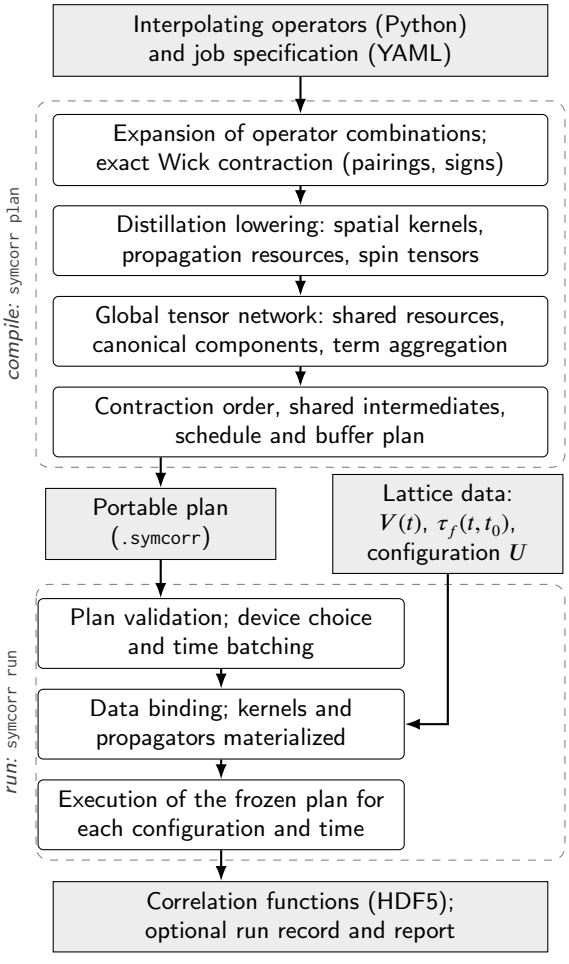


Figure 1: SymCorr workflow. The compile stage depends only on the symbolic operators and the compile settings and produces a portable, hardware-independent plan. Lattice data (Laplacian eigenvectors $V(t)$, perambulators $\tau_f(t,t_0)$ and, for derivative or displaced operators, the gauge configuration U, read as the stout-smeared spatial links used for the Laplacian eigenvectors) and the available hardware enter only at run time.

where α runs over the Wick-contractions "Contr$_\alpha$ [...]" and c_α carries the associated numerical coefficient and the fermionic sign.

The SYMCORR package is designed to separate the physical definition and representation of correlation measurements from the details of their numerical evaluation. The user-provided interpolators determine the Wick contractions and their representation in the distillation space, as illustrated schematically in Eq. 5, while the subsequent numerical procedures, including tensor contractions, redundancy optimizations to avoid repetitive evaluations, memory (de)allocation, and data reuse, are handled by the SYMCORR backend. The physics-level description of the measurements are preserved at every stage of the SYMCORR flowchart to facilitate user-end as well as backend verifications and troubleshooting, while the numerical tensor objects and operations remain blind to the physics that is aimed at. The correlation matrix evaluation is treated within the SYMCORR package as a single structured computational problem, instead of compiling every matrix element as an independent numerical program. This helps in identifying similar algebraic expressions as well as redundant mathematical work across the full calculation and thus providing efficient choices for tensor contraction ordering and data reuse, enabling an optimized workflow. Following the optimization aprogram t the algebraic as well numerical level, SYMCORR also makes educated decisions at the execution stage in regrouping calculations to utilize the provided the hardware resources to its maximum potential. All optimizations, at the algebraic, numerical, and execution stages, are handled by the SYMCORR backend while preserving the underlying physics information. This shields non-expert users from the complexities of optimization, while still giving expert users full freedom to inspect the practical implementation and numerical performance.

We present the SYMCORR workflow in Fig. 1. The user provides the interpolator basis in symbolic form (Sec. 3) together with a job specification that fixes the lattice dimensions, the number of distillation modes, the spin basis, the data locations, and the source and sink times. The calculation then proceeds in two stages. The compile stage uses only the symbolic operators and the compile settings. Linear combinations of operators are expanded into basis correlators, and all quark-line pairings (Wick contractions), their fermionic signs, and their coefficients are generated for every element of the correlation matrix (Sec. 4). The resulting expressions are lowered to the distillation framework, where each Wick term becomes a tensor network of operator-dependent spatial kernels, propagation (perambulator) resources, and spin tensors (Sec. 5). All tensor networks of the requested correlation matrix are then treated as one global contraction problem: identical resources and identical contraction networks are shared across Wick terms and matrix elements, exactly equal terms are combined, a contraction order is chosen, shared intermediates are identified, and the operations are scheduled together with a plan for reusing memory buffers (Sec. 6). The result is saved as a portable plan that depends neither on lattice data nor on hardware. In the run stage, the saved plan is first checked against the current operator definitions and job specification. SYMCORR then chooses a CPU or GPU deployment and a memory-compatible batching of time slices for the available hardware, reads the Laplacian eigenvectors $\{v^{(i)}(t)\}$, the perambulators $\tau_f(t, t_0)$ and, for derivative or displaced operators, the gauge configuration in the form of the stout-smeared spatial links used to construct the eigenvectors, builds the required kernels and propagation tensors, and executes the frozen plan for every gauge configuration and time context. The correlation functions are written to an HDF5 file; a record of the run and a human-readable report can be produced on request.

Now we present the scope of the SYMCORR package. The present implementation can serve in optimized distillation-based correlation matrix evaluations and works on both CPU as well as GPU resources. The package assumes externally supplied operator basis, lattice QCD configuration, associated Laplacian eigenvectors, and the quark perambulators. Operator constructions such as those dealt by packages (MHI) and `OpTion` supplied n Ref. [1, 2], generation of the lattice QCD configurations [**? ?**] and evaluation of Laplacian eigenvectors, and perambulators [**?**] are outside SYMCORR's scope. The package can inspect the available resources such as CPUs, GPUs, memory, and can make educated decisions on efficient execution, however it does not support multi-GPU and MPI based multi-CPU evaluations, which is a straightforward extension. These are limited execution boundaries rather than restrictions on the symbolic definition of the correlator.

At the physics level, the SYMCORR package has been validated for a variety of local as well as bilocal hadron operators, and their full distillation two point correlation matrix evaluations. The results has been cross-verified with independent Chroma based evaluations for rest as well as (multiple) moving frames, for single mesons, single baryons as well as bilocal two-hadron interpolators. Correlation matrices with local four-quark operator and relavant bilocal two meson operators in the T_{1g} have also been tested and cross-verified with an independent realization in the Croma software suite. Beyond this explicitly verified cases, the package has additional capabilities, that includes, but not limited to, multi-hadron correlation matrix evaluations, derivative-based single hadron correlator evaluations, correlation matrices involving disconnected quark loops, three point correlation matrices provided the generalized perambulators [].

3. Symbolic Representation of Interpolating Operators

The SYMCORR package takes in the interpolator basis as a list of ordered algebraic expressions built out of objects representing (anti)quark fields, gauge fields, spin and color structures. At this stage of interpolator definition, the representation is independent of distillation tensors (perambulators and operator kernels/elementals), data layouts, contraction paths, and execution backends.

Quark and antiquark fields of flavor $\mathrm{f} \in \{u, d, c, s, b\}$ are represented schematically as $q_{\mathrm{f}}(x)_{a\alpha} \equiv$ `f("x", "a", "alpha")` and $\bar{q}_f(x)_{a\alpha} \equiv$ `fbar("x", "a", "alpha")`, where x, a, and α denote position, color, and Dirac spin indices, respectively. The explicit flavor information and the quark/antiquark identification in the fermion object names aid in identifying flavor compatibility and in keeping track of fermionic permutation signatures in Wick contractions at the symbolic stage. The literal names chosen for dummy spin, color, and position indices are not part of the mathematical identity of any symbolic expression or the operator built out of it. The package assumes the following convention for the index labels. When a label is first introduced, its index type (spin, color or space) is fixed; attempting to reuse the same label for a different type will be rejected. Reusing the label within the same type defines the intended tensor connectivity of the operator.

Until the backend stage of numerical evaluation, no numerical conventions are assumed for the Dirac gamma structures. At the front end, the user may provide usual gamma matrices to represent the operator spin structures, examples of which will be given later in this section. On the color index, the package supports the Kronecker delta δ_{ab} for the meson-like objects and the antisymmetric Levi-Civita tensor ϵ_{abc} for the baryon-like operators. Spin and color indices follow the usual Einstein repeated index contraction convention within a symbolic expression. A single spin index occurrence remains open and can be used to represent the uncontracted Dirac spin index, such as in baryon operators. A single color index is allowed by the representation but will be reported as a warning as the resulting operator will be gauge dependent. Spin or color indices appearing more than twice are rejected.

Position indices identify the spatial locality associated with fermionic fields within an expression and are therefore not subject to the Einstein summation convention. Two or more fermionic fields carrying the same position label belong to the same locality group, whereas distinct labels can be used to represent multi-local or more general composite operators. This convention helps in assigning a definite three-momentum to each locality group as

$$x_r \longmapsto \mathbf{p}_r, \qquad \mathbf{P} = \sum_{r=1}^{n_l} \mathbf{p}_r, \tag{6}$$

where n_l, $\mathbf{p}_r$ and $\mathbf{P}$ are the number of locality groups within, the three-momentum of locality group labelled r and the total three-momentum carried by the full symbolic expression. In this sense, the position labels represent the momentum orientations within the symbolic expression. Spatial transformations such as pure translations, gauge-covariant parallel transport, covariant derivatives, including the directional information on the fermionic fields are also supported by dedicated objects: `shift`, `transport`, and `derivative`.

Large symbolic expressions that are representative of hadronic operators can thus be constructed as products of elementary fermion field objects, with the factor ordering retained explicitly to ensure faithful evaluation permutation signs during Wick contractions. Momentum information and linear combinations of such expressions can then be assigned to build composite interpolating operators, with further freedom to provide additional metadata. The user-provided operator expressions are subjected to a deterministic index relabeling while preserving the flavor, fermionic and factor ordering, contracted indices and the associated spatial and momentum assignments. This backend relabeling procedure aids in identifying same and similar operator structures. The sink and source operator bases may be provided as separate ordered objects by the user. When an explicit source basis is not supplied, the source operator basis is constructed as the Hermitian conjugate of the sink operator basis.

Now we provide various examples of input operator structures from which more complicated operators can be further built. A generic local meson-like operator is given by a quark bilinear $\mathcal{O}_M(x) = \bar{q}_{f_1}(x)_{a\alpha}\Gamma_{\alpha\beta}q_{f_2}(x)_{a\beta}$. For the isovector pseudoscalar pion operator, this is given by $\mathcal{O}_\pi(x) = \bar{d}(x)\,\gamma_5\,u(x)$, where the spin-color indices are implicit. The input expression for this operator projected to zero momentum at the sink is given by

```
pionsnk = Operator( dbar("x", "a", "alpha")
                   *gamma5("alpha", "beta")
                   *u("x", "a", "beta"),
                    momentum=(0, 0, 0),)
```

The corresponding source operator can be built from the operator definition at the sink as `pionsrc = pionsnk.dagger()`.

The conventionally used local baryon-like operator structure is given by diquark-passenger quark form given by $\mathcal{O}_B(x)_\delta = \epsilon_{abc}[q_1(x)_{a\alpha}\Gamma_{\alpha\beta}q_2(x)_{b\beta}]q_3(x)_{c\delta}$, where Γ carries the spin structure of the diquark, whereas the δ is the Dirac spin index of the passenger quark q_3 as well as the baryon. Simplest such quark trilinear operator for a proton can be build by the assignments $q_1 \to u$, $q_3 \to u$, $q_2 \to d$ and $\Gamma \to C\gamma_5$. The symbolic representation of such a proton operator is given by

```
proton = Operator( epsilon("a", "b", "c")
                    *cgamma5("alpha", "beta")
                    *u("x", "a", "alpha")
                    *d("x", "b", "beta")
                    *u("x", "c", "delta"),
                     momentum=(0, 0, 0),)
```

SymCorr also comes with a convenience constructor for such diquark-passenger quark form of baryon-like operators, using which a proton operator can be built as

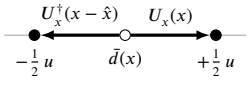
```
proton = build_baryon_operator(
         ["u", "d", "u"], "Cg5",
         momentum=(0, 0, 0),)
```

More complex local hadron operators can further be constructed from these elementary objects and constructions. A local scalar diquark-antidiquark structure representing a compact tetraquark operator can then be represented as

```
dqadq_local = Operator(  epsilon("a", "b", "e")
```

```
pion_dx = Operator(
    dbar("x", "a", "alpha")
    * gamma5("alpha", "beta")
    * derivative(
        u("x", "a", "beta"), "+x"),
    momentum=(0, 0, 0),
)
```

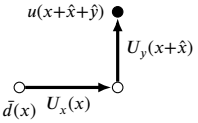
$$\mathcal{O}(t) = \sum_{\mathbf{x}} \bar{d}(x)\,\gamma_5\,(\nabla_x u)(x)$$



(a) derivative ∇_x

```
pion_txy = Operator(
    dbar("x", "a", "alpha")
    * gamma5("alpha", "beta")
    * transport(
        u("x", "a", "beta"), "+x", "+y"),
    momentum=(0, 0, 0),
)
```

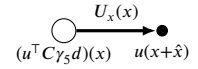
$$\mathcal{O}(t) = \sum_{\mathbf{x}} \bar{d}(x)\,\gamma_5\,U_x(x)U_y(x+\hat{x})\,u(x+\hat{x}+\hat{y})$$



(b) ordered path $(+x,+y)$

```
proton_tx = Operator(
    epsilon("a", "b", "c")
    * spin_matrix("Cg5", "alpha", "beta")
    * u("x", "a", "alpha")
    * d("x", "b", "beta")
    * transport(u("x", "c", "delta"), "+x"),
    momentum=(0, 0, 0),
)
```

$$\mathcal{O}_\delta(t) = \sum_{\mathbf{x}} \epsilon_{abc}\,(u_a^\top C\gamma_5 d_b)(x)\,\left[U_x(x)\,u(x+\hat{x})\right]_{c\delta}$$



(c) spectator-displaced proton

Figure 2: Derivative and displaced operators in the SymCorr language (top), the operators they define (middle), and the lattice structure (bottom; open circles mark the position x of the undisplaced fields, filled circles the displaced quark, arrows the gauge links in the order of the path). The conventions are those of Eq. (**??**); U are the stout-smeared links of the gauge configuration. These three channels, together with the pion operators with ∇_y, ∇_z and the one-link transports T_{+x}, T_{+y}, T_{+z}, are validated against Chroma-LapH in Sec. 7.

```
    * u("x", "a", "alpha")
    * cgamma5("alpha", "beta")
    * d("x", "b", "beta")
    * epsilon("c", "d", "e")
    * ubar("x", "c", "gamma")
    * cgamma5("gamma", "delta")
    * dbar("x", "d", "delta"),
    momentum=(0, 0, 0),)
```

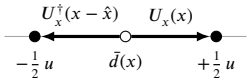
Another example of a simple tetraquark operator is that of a local two-meson structure, which can be represented as

```
two_meson_local = Operator(  dbar("x", "a", "alpha")
                           * gamma5("alpha", "beta")
                           * u("x", "a", "beta")
                           * ubar("x", "b", "gamma")
                           * gamma5("gamma", "delta")
                           * d("x", "b", "delta"),
                           momentum=(0, 0, 0),)
```

Note that in the above construction there are two color singlet quark bilinears forming two independent mesons both featuring in the same locality group leading to a single momentum assignment for the entire structure.

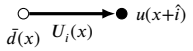
Spatially extended operators can be built by augmenting relevant constituent elementary field as argument to the operator objects: `shift`, `transport`, and `derivative`. Below are three examples of simple meson operators (projected to zero momentum) with nontrivial spatial structures. First one is $\mathcal{O}_{M_1} = \bar{d}(x)\gamma_5\nabla_x(u(x))$ that features a symmetric derivative in the x direction and the corresponding symbolic expression is given by

```
mesondx = Operator(  dbar("x", "a", "al")
                   * gamma5("al", "be")
                   * derivative(u("x", "a", "be"),"+x",),
                   momentum=(0, 0, 0),)
```



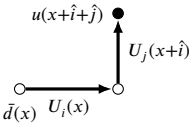
Second operator features a forward gauge-covariant transport along x-direction of the u-quark field $\mathcal{O}_{M_2} = \bar{d}(x)\gamma_5 U_i(x)$ $u(x+\hat{i})$ and the associated symbolic expression is given by

```
mesontx = Operator(  dbar("x", "a", "al")
                   * gamma5("al", "be")
                   * transport(u("x", "a", "be"),"+x",),
                   momentum=(0, 0, 0),)
```



Third operator is a complexified version of second featuring forward gauge-covariant transport along x and y directions of the u-quark field $\mathcal{O}_{M_3} = \bar{d}(x)\gamma_5 U_j(x+\hat{i})U_i(x)u(x+\hat{i}+\hat{j})$ and the associated symbolic expression can be given as

```
mesontxy = Operator(  dbar("x", "a", "al")
                    * gamma5("al", "be")
                  * transport(u("x", "a", "be"),"+x","+y",),
                    momentum=(0, 0, 0),)
```



Note the arguments related to the directions in `transport` follows the order from left to the right. Similarly, explicit gauge links and other color structures can also be introduced as factors in the expression.

Composite operators such as bilocal or multilocal operators could be built as products of simpler structures. Let $H_1(\mathbf{p}_1)$ and $H_2(\mathbf{p}_2)$ be independently defined operators with distinct momentum assignments, their product is given by

$\mathcal{O}_{12} = H_1(\mathbf{p}_1)\, H_2(\mathbf{p}_2)$, where $\mathbf{P} = \mathbf{p}_1 + \mathbf{p}_2$ is the total three-momentum of the operator $\mathcal{O}_{12}$. In the symbolic form, individual expressions for mesons can be concatenated to form the composite operator expression either explicitly or implicitly. In the latter strategy, one defines individual mesons as

```
meson1 = Operator(  dbar("x", "a", "alpha")
                  * gamma5("alpha", "beta")
                  * u("x", "a", "beta"),
                  momentum=(1, 0, 0),)
```

and

```
meson2 = Operator(  ubar("y", "b", "gamma")
                  * gamma5("gamma", "delta")
                  * d("y", "b", "delta"),
                  momentum=(-1, 0, 0),)
```

Following these definitions, the bi-local two-meson operator, where the individual mesons are projected to definite momentum can be represented as

```
two_meson_bilocal = meson1 * meson2
```

Alternatively, one can define the same explicitly as

```
two_meson_bilocal = Operator(  dbar("x", "a", "alpha")
                             * gamma5("alpha", "beta")
                             * u("x", "a", "beta")
                             * ubar("y", "b", "gamma")
                             * gamma5("gamma", "delta")
                             * d("y", "b", "delta"),
                             momenta={"x": (1, 0, 0),
                                    "y": (-1, 0, 0),},)
```

where the individual mesons fall into separate locality groups represented by x and y, which have momentum assignments $\mathbf{p}_x = (1, 0, 0)$ and $\mathbf{p}_y = (-1, 0, 0)$, respectively.

Another important feature supported by the package is the construction of complete composite operators, which are typically linear combinations various expressions $\mathcal{O} = \sum_i c_i\, \mathcal{O}_i$. Addition, subtraction, scalar multiplication, and products of simpler expressions are also accepted as user-defined operator expressions, facilitating such composite operator expression representations. Hermitian conjugation acts on the full symbolic construction, combined with a reversed sign for all the associated momenta. Thus, upon Hermitian conjugation of $\mathcal{O}$, the fermionic field order in each $\mathcal{O}_i$ is reversed and each field object in $\mathcal{O}_i$ is conjugated according to its symbolic rule. Whereas for a linear combination, the coefficients c_i are complex conjugated and each basis operator $\mathcal{O}_i$ is Hermitian conjugated following the rule for product operator. These operations allow a source operator to be derived from a sink definition without manually reconstructing its field ordering, spin structure, or momentum orientation.

4. Quark propagation and Wick contraction

Following the user-input of interpolator basis, symbolic representation of the correlation matrix to be evaluated is instantiated, then all allowed quark propagations are identified, thereafter the correlation matrix is reformed in terms of quark lines or quark propagators. To this end, the operator linear combinations are expanded to build a new correlation matrix for an expanded definite basis of single term operator expressions. This expanded basis aids in identifying redundancies and optimizing evaluation cost and the final correlation matrix in the original basis will be reconstructed at a later stage after symbolic and numerical optimization as well as evaluation stages. The next stage in `SymCorr` is in determining all allowed combinatorial pairing of quark-antiquark pairs for every quark flavor appearing in each of the correlation matrix element in the larger definite basis. For each quark flavor, the number of quark fields should be same as the number of antiquark fields in any given correlation matrix element considered. If this condition fails for any flavor, no contraction term would be evaluated for that matrix element.

Once the quark-antiquark field balance is ensured for each flavor, every bijection between the quark and antiquark fields of the each flavor is identified. n_f quark fields of a given flavor leads to $n_\mathrm{f}!$ possible pairings. Since the choices for different flavors are independent, the complete set of flavor-compatible pairings is given by $N_\mathrm{pair} = \prod_\mathrm{f} n_\mathrm{f}!$. Note that all quark-antiquark fields bijections generated are based on the flavor information explicitly specified in the elementary quark field objects that compose the hadronic operator. For example, u and d remain distinct quark flavors at this stage, even if isospin is assumed to be a good symmetry leading to the use of same light quark propagator/perambulator for either flavors at a later stage of the evaluation. Pairing of quark and antiquark fields of the same flavor when both occur on the same side of a two-point function or within the same composite insertion is also considered as a valid bijection. Retaining all allowed possibilities is necessary to realize the correlations, which may include closed quark loops or disconnected or mixed contraction topologies, to project the desired quantum numbers correctly. All admissible field pairings determined at this stage are retained explicitly and are passed on to the next stage discussed below.

Complete tensor connection within each admissible field pairing leads to a diagram, which is referred to as a Wick contraction diagram and is associated with a unique relative sign. For a given Wick contraction, this sign is fixed by the number parity of the fermionic field reordering required to arrive at the corresponding pairing expression from the original symbolic correlation matrix element. Other tensor objects such as spin matrices, color structures and spatial transformations within the correlation matrix element remain part of the correlator expression, and do not contribute to the fermion permutation signature. Consider a correlation matrix element with N quark fields and thus N antiquark fields, with the original user-defined fermionic field order F. For each given Wick contraction associated with the

correlation matrix element, `SymCorr` builds the fermionic field ordering in which each quark is placed immediately left to its paired antiquark, for example

$$F_{\mathcal{P}} = (q_{i_1}, \bar{q}_{j_1}, \ldots, q_{i_N}, \bar{q}_{j_N}). \tag{7}$$

The fermion permutation sign is given by $s_{\mathcal{P}} = (-1)^{I(\mathcal{P})}$, the number parity of fermion field permutations that maps the original field ordering F to the paired ordering $F_{\mathcal{P}}$.

We demonstrate this exercise with a simple example of correlation function with four fermion fields. Suppose the original fermionic order is $F = (q_1, q_2, \bar{q}_1, \bar{q}_2)$. For the pairing $(q_1, \bar{q}_1), (q_2, \bar{q}_2)$, the paired sequence corresponds to $F_{\mathcal{P}} = (q_1, \bar{q}_1, q_2, \bar{q}_2)$. Mapping F to $F_{\mathcal{P}}$ involves one permutation and hence has sign -1. The only other admissible pairing $(q_1, \bar{q}_2), (q_2, \bar{q}_1)$, the pairing sequence $F_{\mathcal{P}} = (q_1, \bar{q}_2, q_2, \bar{q}_1)$. In this case, mapping F to $F_{\mathcal{P}}$ requires two fermion field permutation and hence has sign +1.

Each Wick contraction, within a correlation matrix element, comes also with an algebraic coefficient $c_{\mathcal{P}}$, which might be originating from the coefficients in the operator linear combinations within in the user-defined basis. These factors are appropriately combined into the coefficient multiplying the logical correlator $c_{\mathcal{P}}^{\text{full}} = s_{\mathcal{P}}\, c_{\mathcal{P}}$. Note that the sign and the algebraic coefficients are kept as separate pieces of the Wick contraction. This aids in making the origin of permutation sign traceable and thus inspectable.

4.1. Canonicalization and symbolic simplification

Canonicalization in SymCorr is used to identify symbolic descriptions that are mathematically equivalent under rules that are explicit in the implementation. It is not used to guess additional field-theory identities. In particular, the frontend retains the expression exactly as supplied by the user, while a separate structural identity removes dependence on arbitrary dummy-index names. Contracted (dummy) spin, color, and position labels are renamed deterministically, and the momentum map is transformed through the same position-label mapping. Open indices keep their names, so that, for example, an operator and its transpose in two open spin indices remain distinct. Ordered factor structure, flavor, transformations, and momentum values remain part of the operator identity, whereas user metadata is treated as annotation rather than symbolic physics.

A second canonicalization step occurs at the compiler boundary before Wick contraction. Field-operation chains are normalized without mutating the frontend operator. For pure coordinate shifts and gauge-covariant parallel transport, adjacent inverse lattice hops cancel. For example,

$$(+x, -x, +y) \longrightarrow (+y). \tag{8}$$

Nested field operations are then fused into a compiler-internal composite operation that preserves their order. Other primitive operations are retained unchanged by this pass, and the operator momentum assignments and metadata are copied to the normalized compiler-side representation.

This normalization is deliberately performed before linear-combination terms are merged. Two frontend operators can therefore remain visibly different as user objects yet become the same compiler basis operator when, for example, their transport paths differ only by adjacent inverse hops. Once normalized, weights multiplying identical ordered basis-operator tuples are added, and a basis term is removed only when its weight cancels exactly; no numerical tolerance is applied. For a two-point correlation matrix, the corresponding simplification pass performs the same coefficient aggregation separately within each matrix element for identical normalized sink–source basis pairs.

The same normalized operator identity is also used at the Wick-cache boundary. The cache key is the complete ordered tuple of basis-operator insertions rather than an unordered set. Consequently, repeated occurrences of the same normalized symbolic correlator can reuse an already generated Wick expansion, while a physically different operator ordering remains a distinct contraction problem. This reuse avoids repeated symbolic work but does not alter the set of Wick pairings or their fermionic signs.

The scope of these transformations is intentionally restricted. The pre-Wick normalizer does not commute or reorder fermion factors, replace spin or color structures using additional identities, infer cyclic trace equivalences, or simplify expressions using numerical tensor values. Such transformations could change the field ordering or require assumptions that belong to later compiler representations. At this stage SymCorr therefore applies only the canonicalization rules for which equivalence is explicit in the symbolic algebra.

There is a separate simplification mechanism later in the compilation pipeline, after the Wick terms have been lowered to numerical contraction graphs. At that stage exact global common-subexpression analysis can canonicalize reusable tensor-network components and combine exactly equal complete Wick terms within a correlator. Like the pre-Wick weights, the later complete-term aggregation removes only exactly cancelling coefficients, and it does so only for scalar terms. Keeping these two stages distinct avoids conflating symbolic algebra with numerical contraction reuse; the latter is discussed in Sec. 6.

4.2. Output of the contraction stage

For each normalized basis correlator, the Wick engine returns an ordered collection of symbolic Wick terms. A convenient schematic description of one such term is

$$W_\alpha = \left(s_\alpha, c_\alpha, \mathcal{P}_\alpha, \mathcal{E}_\alpha, \mathcal{C}_\alpha\right), \tag{9}$$

where s_α is the fermionic sign, c_α is the algebraic coefficient, $\mathcal{P}_\alpha$ is the explicit set of paired quark and antiquark fields, $\mathcal{E}_\alpha$ is the contracted symbolic expression, and $\mathcal{C}_\alpha$ records the locality-graph components inferred from the pairing. These quantities remain symbolic: no perambulator, distillation kernel, tensor shape, external file, memory buffer, or execution device has yet been introduced.

The pairing $\mathcal{P}_\alpha$ is retained directly rather than being recoverable only from a printed algebraic expression. Each entry identifies the quark endpoint and the antiquark endpoint of one logical propagation line, including their flavor and symbolic spin, color, and position labels. This explicit pairing is the authoritative input used by the next compiler boundary to construct logical quark lines. It also retains the information needed to associate field-level derivatives, transports, shifts, and spatial representations with the correct endpoints.

The symbolic expression $\mathcal{E}_\alpha$ retains the non-fermionic tensor content inherited from the original operators and represents contracted quark lines symbolically. In particular, spin matrices, color tensors, gauge-link objects, and their index connectivity remain visible rather than being replaced by numerical arrays at the Wick stage. The later physics-lowering pass reads these symbolic tensors and the explicit pairing together; it does not infer the quark-line topology from numerical tensor shapes.

In addition to the algebraic term, SymCorr computes a locality graph from the pairing. Position labels form the graph vertices and a contracted quark line connects the position labels of its two endpoints. Each resulting graph component records both the locality labels that it contains and the operator insertions touched by those labels. The present Wick-level topology label is a coarse diagnostic: a term is classified as *disconnected* when all components are local to individual operator insertions, *connected* when there are spanning components and no purely local component, and *mixed* when spanning and local components occur together. This terminology does not assert that the complete numerical tensor network consists of one connected component.

For composite operators, the same component records provide an additional structural diagnostic. The current `factorized` property is true when no locality component spans more than two operator insertions; a component touching more than two insertions marks a multi-operator exchange structure. This information is useful for inspecting the generated Wick terms, but it is kept separate from later numerical common-subexpression analysis and contraction planning.

When a basis correlator originates from a linear combination, its previously computed complex basis weight is applied to the Wick-term coefficient while the cached sign, pairing, expression, and component structure remain unchanged. The next lowering step then combines sign and coefficient and translates each Wick term into a backend-independent logical correlator. In the current PhysicsIR, the resulting records contain logical quark lines, spin tensors, color tensors, gauge tensors, field-operation information, and momentum assignments. The Wick-level locality-component and topology diagnostics are not part of this PhysicsIR data contract.

This boundary is important because it marks the end of the purely symbolic Wick problem. By the time a term leaves this stage, SymCorr has fixed which fermion fields contract, the associated Grassmann sign, the algebraic weight, and the symbolic operator structures that accompany those propagation lines. The next stage may reorganize this information into backend-independent physics records, but it is not permitted to invent a new pairing or reinterpret the fermionic sign.

Table 1
Number of Wick terms generated by SymCorr for $C(t,t_0) = \langle \mathcal{O}(t)\,\mathcal{O}^\dagger(t_0)\rangle$, separated into terms whose quark lines all connect source and sink (src–snk), connected terms containing equal-time lines, and disconnected terms with closed loops. The operators are $\pi^+ = \bar{d}\gamma_5 u$, $\bar{u}u$, $p = \epsilon_{abc}(u_a^\top C\gamma_5 d_b)u_c$ and $\Delta^{++} = \epsilon_{abc}(u_a^\top C\gamma_i u_b)u_c$; two-meson and two-baryon operators are products of separately projected hadrons, and $p\pi^+$ is local.

$\mathcal{O}$	terms	src–snk	equal-time	loops
π^+	1	1	0	0
$\bar{u}u$	2	1	0	1
p	2	2	0	0
Δ^{++}	6	6	0	0
$\pi^+\pi^+$	4	4	0	0
$\pi^+\pi^-$	4	1	2	1
$p\pi^+$	12	6	6	0
pp $(I{=}1,\ {}^1S_0)$	48	48	0	0

4.3. Contraction topologies

Because the pairing search is complete within each flavor and is not restricted to source–sink lines, every quark-flow topology allowed by the flavor content of the operators is generated automatically; no diagram is entered by hand or selected by the user. This includes quark lines that start and end on the same time slice and closed quark loops. Representative diagrams are shown in Fig. 3, and Table 1 lists the number of Wick terms that SymCorr generates for representative operators, separated into terms whose quark lines all connect source and sink, terms that contain equal-time lines, and disconnected terms with closed loops. The relative signs and the subsequent tensor-network construction follow from the same rules for all of them.

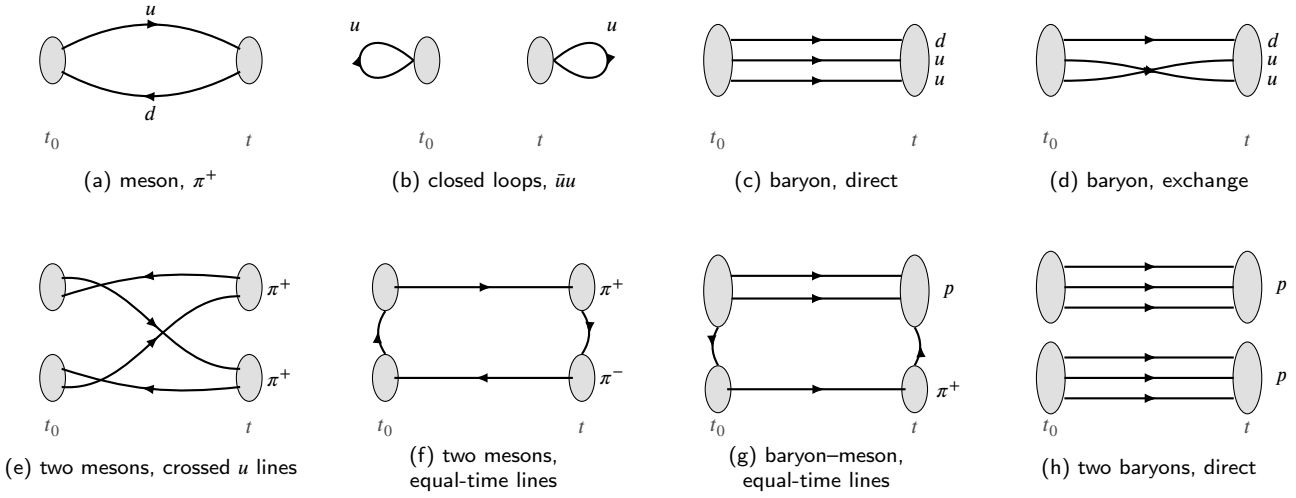


Figure 3: Representative quark-flow topologies generated by SymCorr. Ellipses are hadron operators at the source time t_0 (left) and the sink time t (right); arrows follow quark propagation. (a) Single meson. (b) Disconnected term of a flavor-diagonal meson with a closed quark loop at each time. (c, d) Direct and exchange terms of the proton. (e) Two mesons with crossed lines. (f) Two mesons with one equal-time line at each time (rectangle). (g) Baryon and meson with equal-time lines. (h) Two separately projected baryons. All terms of a correlator, including the remaining exchanges, are enumerated automatically; Table 1 gives the counts.

5. Lowering to the Distillation Representation

5.1. Distillation primitives

The input to distillation lowering is the validated PhysicsIR described at the end of Sec. 4. At this boundary the fermionic pairing and coefficient of each Wick contribution are already fixed. The purpose of the distillation layer is therefore not to redo the field-theory algebra, but to translate the physical records into a small set of objects that describe the spatial/color structures and propagation resources required by a distillation calculation.

The first primitive is a *projected field endpoint*. Every endpoint in PhysicsIR carries an explicit spatial representation, currently either LapH or POINT, together with any field-local derivative, transport, or shift operations. For a LapH endpoint, the compiler realizes the contribution according to the orientation of the fermion field,

$$q_{\mathrm{LAPH}} \longmapsto V, \qquad \bar{q}_{\mathrm{LAPH}} \longmapsto V^{\dagger}, \tag{10}$$

whereas an explicitly POINT endpoint remains a distinct point contribution. This translation records which spatial object is required; it does not load an eigenvector tensor or construct a numerical array. The operations attached to the original endpoint remain associated with the realized field so that their placement is not reconstructed later from tensor shape or ordering.

Before these projected fields are assembled, SymCorr partitions each logical correlator into spatial–color components. The partition is determined from the incidence of field endpoints, position labels, color labels, explicit color tensors, and gauge-link tensors. Each connected component containing at least one field endpoint becomes an operator vertex. Spin tensors do not determine this partition. Consequently, the decomposition is independent of whether the original symbolic object was described as a meson, baryon, tetraquark, or multi-hadron operator; it follows only from the explicit spatial and color connectivity already present in PhysicsIR.

A realized spatial–color component is represented by a `KernelRecipe`. Conceptually, this recipe specifies the operator-dependent projected object that will ultimately be built from the required eigenvector or point contributions, field-local spatial transformations, color tensors, gauge tensors, position variables, and momentum projections. For LapH fields its external spatial legs are LapH-mode indices; POINT fields remain distinguishable and retain a color-like external spatial leg. The recipe is therefore a compiler description of the object to be materialized rather than the materialized tensor itself.

Spin structure is kept as a separate primitive. Gamma matrices and other spin tensors have already been converted into explicit `SpinTensorRecord` objects in PhysicsIR, and they are deliberately excluded from the identity of a spatial/color `KernelRecipe`. This separation mirrors the distillation factorization discussed in Sec. 2: the reusable spatial object and the spin structure that contracts with it need not be treated as one opaque tensor at the compiler boundary. It also allows two contributions with the same spatial/color construction but different spin structure to share the same spatial resource where the later contraction graph permits it.

The remaining primitive class describes quark propagation between these operator-dependent objects. From the exact logical quark lines, later stages reconstruct open fermion paths and closed cycles and assign corresponding propagation requirements. For an open path this requirement is

represented by a `PropagationRecipe`; with no inserted current it denotes an ordinary perambulator requirement, while supported explicit POINT-current insertions can produce a generalized propagation requirement. The detailed construction and current implementation boundary of these resources are discussed below in the subsection on propagation resources.

These objects should be distinguished from concrete numerical data. At this stage a kernel recipe is not yet an evaluated elemental, and a propagation recipe is not an in-memory perambulator. No filename, HDF5 group, source time, tensor shape, physical buffer, CPU/GPU placement, or contraction schedule is chosen by distillation realization. Those choices belong to later graph, resource-binding, materialization, and execution stages. The output of this layer is therefore a more concrete statement of the required distillation calculation while remaining independent of a particular data layout or machine.

5.2. Operator-dependent structures

The operator-dependent part of the distillation calculation is constructed from the spatial–color components identified above rather than from a fixed library of meson, baryon, or tetraquark tensor shapes. For one component A, let $\mathbf{x}_r$ denote its position variables, $\mathbf{p}_r$ the associated momenta, and k_i the external LapH-mode indices carried by its projected fields. A useful schematic form of the corresponding spatial kernel is

$$\mathcal{K}_A(\{k_i\};t) = \sum_{\{\mathbf{x}_r\},\{a_i\}} \exp\left(i\sum_r \mathbf{p}_r\cdot\mathbf{x}_r\right) C_A(\{a_i\}) \times \mathcal{U}_A(\{\mathbf{x}_r\},\{a_i\}) \prod_i \mathcal{V}_i(\mathbf{x}_{r(i)}, a_i, k_i; t), \tag{11}$$

where C_A denotes the explicit color-tensor structure, $\mathcal{U}_A$ denotes any explicit gauge-link structure represented in the component, and $\mathcal{V}_i$ denotes the appropriately oriented LapH factor, including the field-local spatial operations attached to that endpoint. For a quark endpoint this factor is built from V, while for an antiquark endpoint it is built from the conjugate/projected orientation corresponding to $V^\dagger$. Equation (11) is schematic: its purpose is to display the information retained by the compiler rather than to impose one fixed tensor rank or one fixed pattern of spatial indices.

For example, a local bilinear whose two LapH fields are contracted to a color singlet gives a two-mode spatial kernel of the schematic form

$$K^{(\mathbf{p})}_{kl}(t) = \sum_{\mathbf{x},a} e^{+i\mathbf{p}\cdot\mathbf{x}} \left[\mathcal{D}_{\bar{q}} V(t)\right]^*_{\mathbf{x}ak} \left[\mathcal{D}_q V(t)\right]_{\mathbf{x}al}, \tag{12}$$

where $\mathcal{D}_q$ and $\mathcal{D}_{\bar{q}}$ stand for the spatial operations attached to the two field endpoints. The spin matrix is not folded into this kernel. In the simplest separable case the familiar elemental-like object is obtained only after the independent spin structure is attached, for example through a factor $\Gamma_{\alpha\beta} K_{kl}$. This is the concrete reason that SymCorr does not use the full spin–spatial elemental as its reusable spatial resource identity.

The same construction applies without changing the lowering algorithm when the component has higher rank. For a local three-quark color-singlet structure, the spatial part has the schematic form

$$B^{(\mathbf{p})}_{k_1k_2k_3}(t) = \sum_{\mathbf{x},a,b,c} e^{+i\mathbf{p}\cdot\mathbf{x}} \epsilon_{abc} \left[\mathcal{D}_1 V(t)\right]_{\mathbf{x}ak_1} \times \left[\mathcal{D}_2 V(t)\right]_{\mathbf{x}bk_2} \left[\mathcal{D}_3 V(t)\right]_{\mathbf{x}ck_3}. \tag{13}$$

The diquark spin matrix and any remaining open spin index are represented separately. Thus the difference between the bilinear and three-quark examples appears through the number of projected fields and their explicit color connectivity, not through a hadron-specific branch in the distillation compiler.

Momentum is part of the spatial resource itself. SymCorr retains one momentum assignment for each position group in a component and uses the phase convention $e^{+i\mathbf{p}\cdot\mathbf{x}}$; Hermitian conjugation has already reversed the corresponding source momentum where required. For a component containing several independent position labels, the spatial object contains the product of those phase factors, equivalently the exponential shown in Eq. (11). This keeps constituent momenta visible for bilocal and multi-local constructions rather than replacing them by only the total momentum.

The compiler assigns each `KernelRecipe` a deterministic spatial resource identity. That identity is invariant under dummy position/color renaming but includes the projected-field orientations, field-local operations, color tensors, explicit gauge connectivity, and momentum assignments. Spin and quark flavor are deliberately excluded from this spatial identity. Spin appears as separate primitive tensor nodes in the subsequent contraction graph, while the LapH eigenvector subspace itself is a gauge-field spatial object rather than a flavor-dependent quark propagator. Equal spatial resource identities therefore expose possible reuse without yet merging the distinct occurrences that appear in different Wick terms or matrix elements; that global reuse is a later optimization step.

For the currently executable exact-distillation path, the compatibility bridge turns a spatial recipe into primitive operations for the required eigenvector factors, momentum phases, and the supported δ and ϵ color tensors. Derivative information remains attached to the field and can be realized by the numerical tensor provider from derivative eigenvector data or, when configured, from the underlying eigenvectors and gauge links. The recipe layer is broader than this executable subset: explicit `GaugeLink` tensor nodes can be represented in the spatial/color component, but the current numerical kernel ISA rejects such standalone gauge-tensor nodes rather than silently approximating them. This distinction is retained here so that symbolic representability is not presented as equivalent to validated numerical execution.

The result of this lowering is therefore an operator-dependent spatial resource with explicit LapH ports, together with separate spin tensors whose indices are connected only when the primitive contraction graph is built. This is the form in which SymCorr carries the operator structure forward to the propagation and contraction stages: sufficiently concrete to determine the required numerical tensors, but still independent of the contraction order, memory layout, or execution device.

5.3. Propagation resources

After the operator-dependent kernels have been identified, the Wick quark lines must be mapped to the propagation tensors that connect their spatial and spin ports. SymCorr performs this step from the exact fermion topology rather than from operator names or tensor shapes. Each open quark path is decomposed into its ordered Wick-propagator segments and any operator-local transitions that join successive segments. The result is a logical propagation requirement; the compiler still does not load or construct a numerical propagator.

For an ordinary path containing one Wick segment and no absorbed insertion, the required object is the usual distillation perambulator introduced in Sec. 2. The corresponding `PropagationRecipe` records the exact source and sink spatial kernels, the particular open port on each kernel, the flavor carried by the Wick segment, and the spatial representation of the two endpoints. Schematically, its numerical realization has the canonical index structure

$$\tau_f(t_{\mathrm{snk}}, t_{\mathrm{src}}) \equiv \tau_{f;\,k\alpha,\,l\beta}(t_{\mathrm{snk}}, t_{\mathrm{src}}), \tag{14}$$

where (k, α) are the sink LapH-mode and spin indices and (l, β) are the source indices. The orientation is inherited from the Wick pairing and is therefore fixed before any external storage convention is considered.

Open paths can also contain explicit local current insertions. In the current compiler, a current is absorbed into a generalized propagation resource only when its quark and antiquark endpoints are explicitly represented as POINT fields belonging to one spatial–color component. For one insertion, the logical structure is schematically

$$\mathcal{T}^{J}_{f_0 \to f_1}(t_2, t_1, t_0), \tag{15}$$

where the propagation segment before the current carries flavor f_0, the segment after it carries flavor f_1, and J denotes the specified local current structure. SymCorr does not assume that the two flavors are equal. A flavor-changing current is represented as a change between adjacent propagation segments rather than as a flavor change inside one Wick segment.

The generalized recipe retains substantially more information than only the name of the current. For every absorbed insertion it stores the spatial kernel, the exact incoming and outgoing kernel ports, the incoming and outgoing flavors, the endpoint spin labels, and the independent spin tensors connecting those labels. Several POINT bilinear insertions can be retained in path order by the compiler representation. The deterministic propagation-resource identity therefore depends on whether the object is ordinary or generalized, the source and sink representations, the ordered sequence of segment flavors, and the canonical spatial and spin content of the insertions. It deliberately does not depend on a filename, HDF5 dataset name, source time, device, or execution strategy.

This logical identity is separated from the occurrence at which the resource is used. During resource binding, each compiler propagation resource is mapped to one exact external tensor source. The occurrence placement then supplies the frontend source, insertion, and sink operator slots, and the run-time operator-time context converts those slots into concrete Euclidean times. Thus two uses of the same mathematical propagation resource can request different time slices without becoming different compiler resources. Conversely, the binding layer does not use a fallback such as "look up a tensor by flavor"; ordinary and generalized resources are bound by their exact resource identity.

External storage conventions are normalized only when a concrete propagation tensor is materialized. The canonical tensor expected by the contraction graph is ordered as sink mode/spin followed by source mode/spin. For a fixed-source file containing $P_{\mathrm{file}}(t, t_0)$, a direct request with $t_{\mathrm{src}} = t_0$ uses the stored orientation after the configured axis conversion. If instead $t_{\mathrm{snk}} = t_0$, SymCorr reconstructs the reverse propagation using γ_5-Hermiticity,

$$P(t_0, t) = \gamma_5 \, P_{\mathrm{file}}(t, t_0)^{\dagger} \, \gamma_5. \tag{16}$$

The Hermitian adjoint exchanges both mode and spin source/sink spaces; a simple complex conjugation is not equivalent. If neither requested endpoint is the fixed source, that tensor cannot be obtained from the file and materialization fails rather than inventing missing propagation data.

The materialization adapter also owns external source-field and spin-basis conventions. In the validated QUDA-LapH input path, for example, a stored `qdagger` source convention is converted to SymCorr's canonical $\langle q\bar{q} \rangle$ convention using the configured spin basis before the tensor reaches the generic numerical executor. Likewise, a reverse fixed-source transformation is applied in the adapter exactly once. The contraction executor therefore consumes the propagation tensor in the orientation requested by the compiler and does not apply an additional post-hoc conjugation or γ_5 correction.

Closed fermion cycles use a related but distinct lowering. For an exact LapH closed cycle, SymCorr does not request a separately stored disconnected-loop tensor. Instead, every Wick segment is represented by an ordinary perambulator and the successive operator-local spatial and spin tensors close the cyclic contraction. This route currently assumes ordinary all-time distillation perambulators for the required segments. A genuinely POINT disconnected loop cannot

be reconstructed from those perambulators because they do not contain the full point-space all-to-all propagator; such a loop therefore requires additional stochastic, all-to-all, or precomputed-loop information and is rejected by the present exact-distillation lowering.

The compiler and resource-binding layers can consequently represent ordinary perambulators, ordered generalized POINT-current resources, and exact-LapH closed-cycle propagation structure within one topology-driven framework. Their validation status is nevertheless different. The ordinary perambulator path is part of the external-data validation described in Sec. 7, whereas generalized propagation inputs and the broader closed-loop pathways should be regarded as implemented compiler/runtime capabilities unless a separate external numerical validation is stated later in this paper.

5.4. Construction of contraction tasks

Once the operator-dependent kernels, propagation requirements, and spin tensors have been identified, SymCorr converts each complete PhysicsIR Wick contribution into a primitive tensor-contraction graph. This graph is the first compiler representation in which the numerical contraction problem is explicit. It contains the tensors that must participate in the Wick term and the indices that connect them, but it still contains no decision about the order in which those tensors will be contracted.

For a Wick term α, let $T_{\alpha n}$ denote its primitive tensor occurrences and let $I_{\alpha n}$ denote the ordered set of contraction indices attached to tensor n. The graph represents the contribution schematically as

$$C_\alpha = c_\alpha \sum_{\{i\}_{\text{internal}}} \prod_n T_{\alpha n}(I_{\alpha n}), \tag{17}$$

where c_α is the coefficient already fixed in PhysicsIR. Indices that occur on two primitive tensor legs are summed internal indices. An index that occurs on only one leg remains a free output index, so the same graph representation can describe both scalar correlator terms and terms with an explicit open index. The graph validator rejects an index attached to more than two primitive legs rather than assigning an ambiguous multiway contraction meaning.

The current primitive graph has only three tensor kinds: operator-dependent spatial kernels, propagation tensors, and spin tensors. A kernel node exposes one port for every surviving external spatial leg of its `KernelRecipe`. For a LapH kernel these ports become LapH-mode contraction indices. A spin node exposes its left and right spin indices. A propagation node connects the corresponding kernel and spin spaces using the canonical leg order

$$(\text{sink mode}, \text{sink spin}, \text{source mode}, \text{source spin}). \tag{18}$$

The graph therefore records tensor connectivity explicitly rather than relying on a later numerical backend to infer contractions from array shape or node ordering.

Fermion topology determines which propagation nodes are inserted. Along an open fermion path, POINT bilinear transitions that satisfy the generalized-resource requirements described above are absorbed into the corresponding propagation recipe. Their spatial kernel and spin tensors are then omitted as separate graph nodes so that the current is not counted twice. A non-absorbed transition must currently have LapH endpoints and remains explicit through its kernel and spin tensors. Similarly, a standalone POINT spatial kernel is not admitted to the current exact-distillation primitive graph; connected POINT bilinears must be realized through the supported generalized propagation route. Mixed POINT/LapH transitions that cannot be represented by these existing resources are rejected at this boundary rather than assigned a guessed tensor realization.

Each primitive tensor occurrence carries two different identities. The *node identity* distinguishes the occurrence inside this particular Wick term, while the *resource identity* describes the reusable mathematical tensor that the occurrence requires. Two kernel occurrences, for example, may have the same spatial resource identity but different node identities because they occupy different places in the term. The primitive-graph stage intentionally preserves both occurrences. No deduplication is performed merely because their resource identifiers agree.

This distinction is necessary because resource equality and tensor occurrence are different statements. A Wick term may use the same perambulator resource more than once, and those two uses must retain distinct contractions even though they may ultimately be materialized from the same external data source. The same applies to repeated spatial kernels and spin matrices. Occurrence-specific ports and index bindings therefore remain attached to each node until a later global analysis proves what can be reused without changing the contraction.

The graph is validated before it is passed to optimization. Node identifiers must be unique, a tensor cannot expose the same local port twice, contraction indices cannot be empty, and no global index may have degree greater than two. A graph in which every index has degree two is a closed scalar contraction; degree-one indices define its external interface. These checks establish the well-formed numerical tensor network independently of any proposed contraction path.

SymCorr constructs one such `WickContractionGraph` for every complete PhysicsIR Wick term. Equal primitive resources therefore remain visible across terms and across correlation-matrix elements, but they have not yet been combined. The next stage operates on this collection globally: it deduplicates exactly equal resource requirements, canonicalizes reusable connected tensor components, and aggregates exactly equal complete terms before contraction-order selection. Those operations belong to the execution and optimization framework in Sec. 6; they are deliberately separated from the task construction described here.

At the end of this subsection the physics lowering is complete. The compiler has fixed the coefficient, the required

primitive tensor resources, and their exact index connectivity. It has not chosen pairwise contraction order, common intermediate expressions, execution schedule, memory buffers, hardware device, or numerical tensor values. Those remaining decisions can therefore optimize an already fixed tensor problem without modifying the physical correlator.

6. Execution and Optimization Framework

6.1. Backend-independent execution representation

The execution layer begins from the collection of primitive Wick-contraction graphs constructed in Sec. 5. Its first purpose is to turn those term-local tensor networks into one hardware-independent representation of the complete requested calculation. The representation is progressively refined through global exact reuse, contraction-order selection, partial-intermediate reuse, and finally an explicit logical dependency graph. At every one of these stages the compiler is still describing mathematical values and operations rather than a schedule for a particular CPU or GPU.

The first global representation is a `GlobalComputationDAG`. Primitive resource requirements are identified by exact resource identity, while tensor occurrences retain their operator-slot placement. Connected pieces of the primitive tensor network are canonicalized independently of temporary node names and dummy contraction-index labels. One canonical component computation can therefore be referenced by several Wick terms or several correlation-matrix elements without treating occurrence-specific placement as part of the reusable resource itself. The requested correlators remain explicit as sums of Wick terms, so global reuse does not collapse distinct physical outputs.

Each unique connected component is then assigned a backend-independent contraction plan. SymCorr presently delegates the algebraic path search to `opt_einsum`, but stores the result in its own compiler representation. A plan is an ordered sequence of contraction or reduction steps; every step records its input values and interfaces, the indices eliminated at that step, the output interface, an estimated operation count, and the size of the resulting intermediate. This separates the mathematical contraction tree from the library used to discover it and leaves room for a different path optimizer without changing the surrounding compiler stages.

Contraction-path search requires numerical dimensions in order to estimate cost. In the current exact-distillation model, LapH-mode indices have dimension N_v for the job and spin indices have dimension four. These dimensions are provided through a size model used for path selection,

$$|i| = \begin{cases} N_v, & i \in \text{LapH-mode space}, \\ 4, & i \in \text{Dirac-spin space}. \end{cases} \tag{19}$$

They are optimization inputs rather than part of tensor-resource identity. A change in N_v, or in the path-search strategy, can therefore change the chosen contraction order without changing the symbolic correlator or its primitive tensor connectivity.

Once a contraction tree has been selected, SymCorr rewrites its intermediate values into a second global DAG. Primitive tensor values are distinguished by both reusable resource identity and exact logical placement, while contraction subtrees are canonicalized from their recursively defined inputs and interfaces. Temporary component step names and dummy local indices are excluded from this identity. As a result, two algebraically identical partial contractions can refer to the same logical intermediate even when they arose in different component plans. Importantly, this pass does not alter the contraction order that was already selected; it only recognizes identical values inside those frozen trees.

The final compiler representation before scheduling is the `LogicalExecutionDAG`. It makes every remaining operation explicit. Its nodes fall into four classes:

$$\begin{aligned} v_r &= \text{EvalPrimitive}(r; \pi), \\ v_m &= \text{Contract}(v_{a_1}, \dots, v_{a_k}), \\ w_\alpha &= c_\alpha \prod_j v_{\alpha j}, \\ C_i &= \sum_{\alpha \in i} w_\alpha, \end{aligned} \tag{20}$$

where r is a primitive resource, π is its logical operator placement, v_m is one reusable contraction intermediate, w_α is one Wick-term product, and C_i is one requested correlator output. Repeated uses of the same value remain distinct occurrences in an operation when multiplicity is part of the algebra, for example when the same disconnected scalar component appears twice in one Wick term.

The logical DAG therefore answers a precise question: *what values must be produced, and which earlier values does each operation depend on?* Primitive nodes have no dependencies; contraction nodes depend on their input values; Wick-term nodes multiply their component results by the already fixed coefficient; correlator-sum nodes depend on the contributing Wick terms. The output map identifies the final node corresponding to every requested correlator. Dependency cycles, missing values, incompatible value interfaces, and inconsistent Wick-term or correlator output indices are rejected by the logical-DAG validator.

A backend-neutral static analysis can subsequently derive exact logical tensor metadata from this DAG. Mode and spin index spaces remain explicit even when they happen to have the same numerical dimension, and each value receives its logical shape, element count, scalar type, byte estimate, producer/consumer relationships, fanout, and distance to an output. This analysis still describes one logical value at one exact placement; time batching, data residency, file layout, and physical memory slots are deliberately absent.

No representation described in this subsection is a physical execution plan. In particular, a topological ordering of the logical DAG merely demonstrates that its dependencies are acyclic; it is not the selected runtime schedule.

Likewise, estimated FLOPs and intermediate sizes used during contraction-path search do not choose a device or allocate memory. Scheduling, lifetime-aware buffer reuse, time-context batching, staging, and hardware feasibility are introduced only by the later planning stages described below. This boundary allows those deployment choices to change while preserving the same optimized mathematical execution DAG.

6.2. Reuse of repeated structures

A central capability of the SymCorr framework is the identification and elimination of redundant computational work across an entire correlation matrix. Rather than treating each matrix element $C_{ij}(t - t_0)$ and each contributing Wick term as an independent contraction program, SymCorr formulates the global calculation as a unified computation directed acyclic graph (`GlobalComputationDAG`). This subsection explains how Wick-contracted terms are represented as attributed graphs, the multi-tier identifier hierarchy used throughout the compiler, the graph-isomorphism algorithm that discovers topologically equivalent contractions, and how this optimization operates in practical hadron correlation matrices.

6.2.1. Graph representation of hadron contractions: The proton example

To illustrate the graph formulation concretely, consider the rest-frame, positive-parity nucleon (proton) interpolating operator,

$$\mathcal{O}_N(t) = \epsilon_{abc} \left[u^a(t)^\top C\gamma_5 d^b(t)\right] P_+ u^c(t), \tag{21}$$

where $C = \gamma_2\gamma_4$ is the charge-conjugation matrix, ϵ_{abc} is the totally antisymmetric $SU(3)$ color tensor, and $P_+ = \frac{1}{2}(\mathbb{I}+\gamma_4)$ is the positive-parity energy projector. The corresponding Euclidean two-point correlation function is

$$C_{NN}(t, t_0) = \left\langle \mathcal{O}_N(t)\,\overline{\mathcal{O}}_N(t_0) \right\rangle, \tag{22}$$

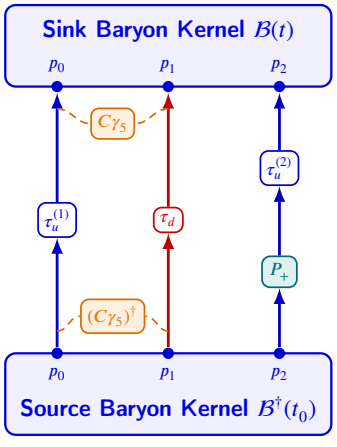
where $\overline{\mathcal{O}}_N(t_0) = \mathcal{O}_N(t_0)^\dagger\gamma_4$ denotes the adjoint source operator.

Because the proton contains two valence up quarks of identical flavor, Wick's theorem generates two distinct contraction terms ($N_u! \times N_d! = 2! \times 1! = 2$):

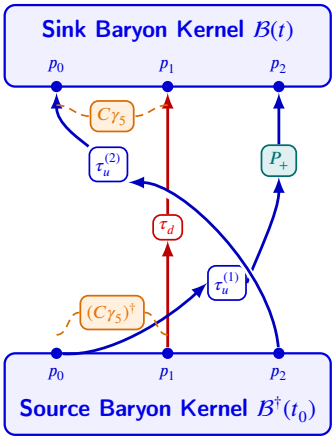
1. **Direct term**: with fermionic permutation sign $+1$, where up quark 1 at the source propagates into up quark 1 inside the sink diquark, while up quark 2 propagates as the spectator quark.
2. **Exchange term**: with fermionic permutation sign -1, where the two up-quark lines cross: up quark 1 at the source propagates into the sink spectator, while up quark 2 propagates into the sink diquark.

In SymCorr, each Wick term is converted into an explicit tensor network graph (`WickContractionGraph`), displayed schematically in Fig. 4.

- **Nodes**: Each graph contains 12 primitive tensor nodes:
  - Two spatial baryon kernels (`kernel_0` at the sink and `kernel_1` at the source), each carrying three external LapH-mode ports: p_0, p_1, p_2.
  - Three quark perambulators $(\tau_u^{(1)}, \tau_d, \tau_u^{(2)})$, each carrying four ports: sink mode, sink spin, source mode, and source spin.
  - Seven Dirac spin tensors representing the source/sink $C\gamma_5$, P_+, and spinor conjugation structures.
- **Edges**: Edges correspond to contracted dummy indices (degree 2). A mode edge connects a perambulator mode port to a baryon kernel mode port. A spin edge connects a perambulator spin port to a Dirac matrix.



(a) Direct contraction graph (sign $+1$)



(b) Exchange contraction graph (sign -1)

Figure 4: Graph representations of the two proton Wick contraction topologies in SymCorr, following the convention of distillation hadron contraction networks [5]. Blue and red solid directed lines represent up- and down-quark perambulator propagations (τ_u, τ_d) connecting source ports to sink ports. Dashed orange links represent Dirac spinor contractions with diquark matrices $C\gamma_5$ and positive-parity energy projectors P_+.

6.2.2. Identifier taxonomy in the compiler

To manage data reuse, compilation, and execution without name clashes or ambiguous sharing, SYMCORR maintains four strictly decoupled tiers of identifiers:

1. **`correlator_id`**: Identifies the high-level physical correlator requested by the user in the job declaration (e.g., `c_proton` or `C_0_1`).
2. **`node_id`**: A diagram-local occurrence identifier assigned to a specific tensor node within one un-optimized Wick graph (e.g., `kernel_0`, `propagation_1`, `spin_3`).
3. **`resource_id`**: A globally unique identifier for the underlying physical/numerical data source. It is generated from a deterministic SHA-256 hash of the mathematical specification (e.g., `spatial_kernel_6427fc17...` and `spatial_kernel_63b068c2...` for the two proton baryon blocks, one built from V at the sink and one from $V^\dagger$ at the source, and `perambulator_cfa5757f...` and `perambulator_b3fea9be...` for the up- and down-quark perambulators; the values are those of the present version). The direct and exchange graphs have distinct node occurrences but share exactly these `resource_id` values.
4. **`computation_id`** (or **`component_id`**): Identifies a unique, canonicalized, connected tensor contraction network (e.g., `component_9b66414a...` for the direct graph and `component_301a5cf4...` for the exchange graph). This identifier is invariant under dummy index renaming and node reordering.

6.2.3. Graph isomorphism: Identifying identical contractions through canonical labeling

Two contraction graphs may represent the exact same mathematical contraction, yet appear entirely distinct to a computer. This occurs for two simple reasons:

1. **Dummy index naming**: Consider two mathematically identical contractions,

$$\sum_{\alpha} \tau_{\alpha\beta} \, (C\gamma_5)_{\beta\alpha} \quad \Longleftrightarrow \quad \sum_{\mu} (C\gamma_5)_{\nu\mu} \, \tau_{\mu\nu}.$$

   Although they compute the same tensor value, an automated parser sees different dummy labels (α, β versus μ, ν).
2. **Arbitrary node order**: Tensors in memory can appear in any sequence (listing τ before $C\gamma_5$ or vice versa) without altering the physical contraction result.

To recognize identical contractions across different correlators, SYMCORR converts every contraction graph into a unique, standardized form—a process known as *canonical labeling*.

Redstar [5] performs this step with the general-purpose canonical-labeling package `nauty`. Lattice QCD contraction graphs are, however, compact, typically containing only 5–25 tensor nodes, so that an exhaustive search over starting nodes is affordable. SYMCORR therefore uses a lightweight, self-contained canonicalization strategy based on three intuitive steps:

- **Step 1: Graph traversal and index standardization**: Starting from a chosen tensor, the compiler walks through the graph by following port connections in a fixed numerical port order (port 0, port 1, ...). As internal connections and open indices are encountered along the walk, they are renamed systematically to sequential canonical labels ($i_0, i_1, i_2, \ldots$ for contracted internal indices and $o_0, o_1, \ldots$ for open external indices).
- **Step 2: Unbiased root selection**: Because a loop of contracted tensors has no natural "first" node, SYMCORR tries starting the walk from every candidate tensor in the graph. Each choice of starting node produces a standardized candidate string describing the walk.
- **Step 3: Deterministic tie-breaking (Lexicographical minimum)**: Among all candidate description strings generated across all possible starting nodes, the compiler deterministically selects the one that is alphabetically earliest (the lexicographical minimum).

Algorithm 1 Canonical Graph Isomorphism in SYMCORR

Require: Contraction graph G with tensor nodes, port definitions, and index connections.
Ensure: Canonical identifier `computation_id` and standardized contraction plan.

1: **Initialize** candidate list $C \leftarrow [\,]$.
2: **for** each candidate starting tensor r in G **do**
3: Walk through connected tensors starting from r, following port connections in fixed numerical port order.
4: Record the sequential visit order of tensors $(t_0, t_1, \ldots)$.
5: Encode tensor kinds, underlying resource IDs, and relative edge steps into a standardized string S_r.
6: Add S_r to candidate list C.
7: **end for**
8: **Select** the alphabetically earliest candidate:

$$S_{\min} \leftarrow \min(C)$$

9: **Standardize indices**: Rename internal contracted indices to $\{i_0, i_1, i_2, \ldots\}$ and open indices to $\{o_0, o_1, \ldots\}$ according to the winning walk.
10: **Generate fingerprint**: Compute the cryptographic hash:

$$\texttt{computation_id} \leftarrow \mathrm{SHA{-}256}(S_{\min})[:20]$$

11: **return** Standardized contraction component and its `computation_id`.

Because this procedure exhaustively tests all starting tensors and deterministically selects the lexicographically minimal description, the resulting string $S_{\min}$ and its hash `computation_id` serve as a canonical fingerprint: two contraction graphs receive the same `computation_id` exactly when

they are identical up to the order of their tensors and the names of their indices, with the same tensor kinds, resources and port connections. Whenever two hashes agree, the full strings are compared, so that a hash collision cannot merge different graphs. Equalities that rely on tensor symmetries or on physics relations such as Hermiticity are not detected by this structural test.

Once graphs are fingerprinted in this way, the compiler tests for graph equivalence with a single $O(1)$ hash lookup. If two graphs share the same `computation_id`, the tensor contraction is scheduled and computed only once, and the result is reused wherever needed in the global correlation matrix.

6.2.4. Reuse in a nucleon correlation matrix

Where does graph isomorphism provide practical computational reuse in realistic lattice QCD spectroscopy? Consider a 2×2 correlation matrix of spin-$1/2^+$ rest-frame nucleon interpolators constructed from two distinct diquark structures: a pseudoscalar diquark operator $\mathcal{O}_1(t)$ and an axial-vector/temporal diquark operator $\mathcal{O}_2(t)$,

$$\mathcal{O}_1(t) = \epsilon_{abc} \left[u^a(t)^\top C\gamma_5 d^b(t)\right] P_+ u^c(t), \tag{23}$$

$$\mathcal{O}_2(t) = \epsilon_{abc} \left[u^a(t)^\top C\gamma_4\gamma_5 d^b(t)\right] P_+ u^c(t). \tag{24}$$

The four matrix elements evaluated across the correlation matrix are

$$\mathbf{C}(t, t_0) = \begin{pmatrix} C_{11}(t, t_0) & C_{12}(t, t_0) \\ C_{21}(t, t_0) & C_{22}(t, t_0) \end{pmatrix}, \tag{25}$$

where each correlator element is defined by $C_{ij}(t, t_0) = \langle \mathcal{O}_i(t)\, \overline{\mathcal{O}}_j(t_0) \rangle$.

Each of the four correlators contains two Wick terms, so the matrix produces eight contraction graphs with 96 tensor occurrences in total. The global compiler pass identifies the following structure:

1. **Shared tensors across the full basis**: Because spin matrices are not part of the spatial kernels, all eight graphs use the same two baryon kernels, one built from V for the sink block and one from $V^\dagger$ for the source block, and the same up- and down-quark perambulators. Together with seven distinct spin matrices, the 96 tensor occurrences require only 11 distinct tensors, each of which is loaded or built once per time slice.
2. **Distinct contraction networks**: In this example all eight networks are distinct. In particular, the off-diagonal elements C_{12} and C_{21}, which are related by Hermiticity on the gauge ensemble, carry different spin matrices at the sink and the source ($C\gamma_5$ at the sink and the conjugate of $C\gamma_4\gamma_5$ at the source, or vice versa). The structural test of Algorithm 1 therefore treats them, correctly, as different computations. Exploiting relations of this kind would require an additional, separately validated rule and is not done.
3. **Exact algebraic cancellation (zero pruning)**: Within a single correlator sum, if two contributing scalar terms possess identical canonical components but opposite coefficients ($c_\alpha + c_\beta = 0$), SymCorr sums the coefficients and prunes the term from the execution DAG, so that no operations or buffer memory are spent on vanishing contributions.

Reuse of identical networks becomes important when the same operator structures recur across many matrix elements and Wick terms, and partial contractions shared between different networks provide a further reduction (Sec. 6). The reductions obtained for larger systems are quantified in Sec. 9.

6.3. Contraction-path selection

The order in which the tensors of one connected component are contracted does not change its value, but it determines the operation count, the size of the intermediates and, through partial-intermediate reuse, how much work different components can share. SymCorr exposes this choice as one compile option, `compile.contraction.strategy`:

`greedy` (default) passes every unique component to the greedy path search of `opt_einsum`. It is fast and deterministic, but it chooses each pairwise step from local cost information only.

`dynamic-programming`, `auto-hq` use the corresponding `opt_einsum` searches. Dynamic programming minimizes the operation count of one component exactly; `auto-hq` uses an exhaustive search for up to five tensors, dynamic programming for up to sixteen and the best of 128 seeded randomized-greedy trials beyond that. Both are deterministic for a fixed problem, but their compile time grows rapidly with the number of tensors in a component.

`reuse-aware` is a SymCorr meta-strategy. It builds the complete plan four times, with greedy, dynamic-programming and one- and two-branch searches, and evaluates every candidate *after* partial-intermediate reuse, logical scheduling and physical buffer assignment. The candidate with the fewest scheduled FLOPs is selected among those whose buffer pool does not exceed a ceiling. Without an explicit byte budget the ceiling is the buffer capacity of the greedy plan, so the selection never increases memory relative to the default; `compile.physical.memory_limit_bytes` replaces it by a hard user limit. The exact searches are applied only when every component has at most twelve tensor inputs; otherwise the whole calculation keeps the greedy candidate so that compile time stays predictable.

`adaptive` stores the deterministic greedy baseline and lets the runtime choose between it and the reuse-aware candidates by timing them on the selected device with synthetic tensors of the exact planned shapes. The chosen execution payload receives its own hash, so resumable output never mixes schedules.

Independently of the strategy, `compile.contraction.memory_limit` bounds the size of every pairwise intermediate (in tensor elements) during path search. It is a path constraint, not a buffer budget, and can lower the operation count as well as the memory when the unconstrained greedy search builds a large intermediate early. Section 9 quantifies the effect of these choices for the benchmark systems.

6.4. Memory and time-slice planning

After path selection the logical DAG still describes *what* must be computed. Two compile-time passes decide *when* each value exists and *where* it is stored; a third, run-time pass decides how many independent time slices are processed together.

Logical schedule and lifetimes A list scheduler chooses one legal sequential order of all logical operations. The default `memory_aware` policy ranks ready operations by downstream criticality, value fan-out, tensor sizes, contraction cost and, in particular, by how much input memory an operation releases once it has been executed; `critical_path` and `topological` orders are available for comparison. The schedule fixes, for every value, the step at which it is produced and the last step at which it is consumed. Assuming that an operation output is materialized before its inputs are released, this yields the logical peak of simultaneously live bytes.

Physical buffers The physical planner maps scheduled values onto reusable abstract buffer slots. Two values may share a slot only when their lifetimes do not overlap. The default `best_fit` policy places a value into the smallest released slot that is large enough; `size_matched` reuses only equal-sized slots, which makes the physical peak equal to the logical peak for the same schedule at the price of more total capacity on some workloads; `none` disables reuse. Requested correlator outputs are retained until the end of the work unit. The resulting plan therefore carries four memory figures that are reported without touching any lattice data: the total size of all materialized values (no reuse), the logical peak live bytes, the physical peak active bytes and the total reserved buffer capacity. Because a resident executor allocates the buffer pool up front and produces each operation result before copying it into its slot, the exactly modeled resident requirement of one work unit is

$$M_{\mathrm{known}} = M_{\mathrm{pool}} + \max_{v} M_v, \tag{26}$$

where the maximum runs over all produced values.

Work units and time batching The compiled plan describes one *work unit*: one gauge configuration and one time context, i.e. one assignment of times to the operator insertions (one source–sink pair (t_0, t) for a two-point function). The runtime may stack B work units of the same configuration into one execution. The batched executor prepends one private axis to every primitive, intermediate, Wick-term and correlator value; this axis is present in every operation output and is never contracted, so different time contexts cannot couple. For $B > 1$ the modeled resident peak is

$$M_B = B\, M_{\mathrm{pool}} + \max\bigl(B\, M_{\mathrm{transient}},\ 2B\, M_{\mathrm{primitive,max}}\bigr), \tag{27}$$

where the second term covers either the largest batched operation result or the temporary coexistence of per-context primitive tensors with their stacked copy. With `run.time_batch_size: auto` (the default) the candidates are powers of two and the number of pending time contexts, limited by the safe memory of the selected device (Sec. 6.5). The feasible candidates are timed with seeded synthetic tensors of the exact planned shapes, subject to guards of 16 GiB of synthetic data, 10^{13} scheduled FLOPs per sample and an estimated 180 s calibration budget, and the size with the smallest median time per work unit is chosen. Results of the batched and unbatched executions are compared during calibration. Batch size one is always available and is used whenever memory cannot be measured or calibration is incomplete; an explicit integer batch size is a reproducibility override that is still checked against the current memory. Configurations are never mixed in one batch, and a final short batch is allowed.

Configuration-scoped reuse Within one gauge configuration the runtime allocates one buffer pool at the selected maximum batch size and reuses it for all batches; a short final batch activates a prefix of the existing pool. Primitive tensors whose complete numerical request agrees (for example the source-time kernels, which are the same for every sink time) are retained in a bounded cache whose admission budget is the remaining safe memory after the batch peak of Eq. (27). Read-only HDF5 handles are shared, a batch of perambulator requests along one time axis is coalesced into one covering slab read, and the reads for the next batch can be staged by one background reader (with pinned host memory and a separate stream on CUDA) while the current batch is contracted. All of these are run-time lifetime optimizations: they change neither the mathematical reuse decided by the compiler nor the serialized plan nor the resume identity, and their admission decisions and hit counts are reported in the run summary.

6.5. Hardware-aware execution planning

Hardware information enters only at run time and never enters symbolic expressions, resource identities, reuse decisions or the plan fingerprint. A read-only snapshot records the memory visible to the process (host memory capped by a cgroup limit when one applies, CPU count respecting process affinity, and for every visible CUDA device its current free memory including allocator cache that the process can reuse). From the available memory a safety reserve is retained, the larger of a fraction and an absolute minimum,

$$M_{\mathrm{safe}} = M_{\mathrm{avail}} - \max\bigl(f\, M_{\mathrm{avail}},\ M_{\mathrm{min}}\bigr), \tag{28}$$

with $(f, M_{\min}) = (0.20, 2\ \text{GiB})$ for host memory and $(0.10, 1\ \text{GiB})$ for a CUDA device. The reserve covers the operating system, allocator overhead, provider-owned tensors and library workspaces, which are not modeled exactly. All defaults form one immutable policy object that applications can replace.

With `run.device: auto` the planner prefers a fully resident CUDA deployment when Eq. (26) fits into the safe device memory. If it does not, but the working set fits into safe host memory, a *host-staged* plan is derived: complete values stay in a host buffer pool, and each contraction copies bounded operand tiles to the device and writes the result tile directly into its host buffer. Only LapH-mode indices may be split; spin indices are never tiled. Tiles of an output index fill disjoint slices, whereas tiles of a contracted index are accumulated before the Wick-term coefficient is applied, so the modeled device memory of one operation is its input tiles plus one output tile (two when a contracted index is split). If neither CUDA deployment is safe, a resident CPU deployment is used. An explicitly requested CUDA device may use host staging but never falls back to another device. The selected device is revalidated before every unfinished work unit; because completed work units are already on disk, a failure at that point can be resumed. Host staging trades device memory for more kernel launches and transfers and is a feasibility mechanism rather than a speed optimization; time batching is not combined with it.

6.6. Numerical execution

Binding and materialization For each work unit the compiled primitive resources are bound to data. Spatial kernels (meson elementals, color-ϵ baryon kernels, and their derivative or gauge-transported variants) are built from the LapH eigenvectors and, when needed, the smeared gauge links of the configuration; perambulators are read from HDF5 in either fixed-source or all-time layout and canonicalized to (sink mode, sink spin, source mode, source spin), with the reverse orientation reconstructed by γ_5-Hermiticity when only the forward direction is stored. A binding plan prepared for one work unit is reused for the next only when the physical plan, spin basis, propagation sources, kernel providers and operator-time pattern agree exactly; otherwise execution fails instead of inferring a replacement. Shapes, scalar types and buffer capacities are validated before any contraction.

Contraction The executor processes the stored physical operations in their scheduled order using PyTorch tensors on the CPU or on one CUDA device. It interprets only tensors and index labels; flavor, operator names and propagation physics are no longer visible at this level, and no contraction order is re-optimized at run time. An exact structural recognizer maps eligible binary contractions to a single `torch.mm` or `torch.bmm` call that writes directly into the planned output buffer; all other contractions use a fixed-path `opt_einsum` expression cached by labeled shape. Wick-term coefficients are applied to the component products, and the correlator sums are accumulated in their output buffers.

Output and resumption Results are written to one HDF5 file per job. Each requested correlator is a dataset of shape $(N_{\text{cfg}}, N_{\text{time}})\times$ (its open spin and other output axes), chunked by work unit and annotated with its correlator identifier and axis labels. A Boolean `completed` array of shape $(N_{\text{cfg}}, N_{\text{time}})$ is set only after all correlators of a work unit have been written and flushed. The file also stores the plan hash, a run fingerprint and the job metadata, so an interrupted job resumes with the unfinished work units only and refuses to append to a file produced by a different plan or run configuration. A machine-readable run record separates the time spent in run-time planning, data preparation, numerical execution, input waits and output writing; these records are the source of the timings in Sec. 9.

7. Validation and Correctness

7.1. Validation strategy

A correlator generator can fail in ways that are invisible in a single number: a wrong relative sign between two Wick terms, a transposed spin matrix, a complex-conjugated phase, or a mismatched storage axis can all produce smooth, plausible-looking correlators. We therefore distinguish three kinds of reference, in decreasing order of independence:

1. *external code*: correlators or LapH elementals produced by Chroma/Chroma-LapH for the same gauge configuration, eigenvectors and perambulators;
2. *independent contractions*: NumPy programs that enumerate the ordered Grassmann pairings themselves, build the LapH elementals directly from the eigenvectors and contract them with `einsum`, without importing the SymCorr Wick generator, compiler or executor;
3. *exact identities*: relations between *differently* contracted correlators that must hold configuration by configuration, such as flavor symmetry when all light flavors are bound to the same perambulator, or the vanishing of a Pauli-forbidden channel.

All comparisons use the full complex correlator, and for baryons the full unprojected 4×4 spin matrix rather than a parity trace. Each comparison also carries a negative control (opposite momentum phase, flipped coefficient, omitted disconnected term, or opposite derivative sign) that must fail, so that agreement is not an artifact of a loose tolerance around small numbers. The real-data checks use the A653-like $N_f = 3$ ensemble described in Sec. 9.1, except for the comparisons with Chroma correlator files (single mesons, bilocal two-meson and local four-quark operators), which use a separate Chroma data set.

7.2. Symbolic and Wick-contraction tests

The portable regression suite (1524 tests, of which six require a CUDA device) exercises the symbolic layer directly: flavor-compatible pairings, the permutation-parity sign of every pairing, the coefficients of linear combinations, exact cancellation of terms with identical canonical form and opposite coefficients, the normalization of spin matrices in

Table 2
Real-data validation of SymCorr correlators. "Deviation" is the maximum absolute difference, or, where marked, the maximum difference divided by the largest reference magnitude. All comparisons use complex128 arithmetic.

System	Reference	N_v	Deviation	Remarks
D, K, D^*, K^* (rest frame)	Chroma correlators	–	$\leq 1.2 \times 10^{-12}$	all 24 time slices
D, $\vec{P} = (0,0,1)$	Chroma correlators	–	1.3×10^{-13}	opposite phase fails
bilocal two-meson, 13 channels	Chroma correlators	–	$\leq 6.4 \times 10^{-12}$	direct and exchange terms
$\bar{d}\gamma_5 D_i u$, $\bar{d}\gamma_5 T_{+i} u$, $T_{+x,+y}$	Chroma-LapH elementals	64	1.0×10^{-14} (rel.)	7 channels, 4 sink times
$\epsilon(u^T C\gamma_5 d)\, T_{+x} u$	transport reference	64	1.8×10^{-16} (rel.)	full 4×4 spin matrix
local four-quark T_1^+	Chroma correlators	45	2×10^{-14} (rel.)	up to normalization V^2
$p\pi^+$, $I = \frac{1}{2}$ $N\pi$, $N\sigma$	NumPy contraction	8	$\leq 5.4 \times 10^{-15}$ (rel.)	incl. equal-time propagation
$N\pi\pi$ ($I = 0$ pair)	NumPy contraction	3	6.3×10^{-15} (rel.)	6624 Wick terms per sink
Δ^{++}, 3×3 vector-spin	NumPy contraction	8	2.2×10^{-15} (rel.)	before and after projection
scalar, γ_4 loops; $\bar{u}u$ two-point	NumPy contraction	64	$\leq 4.6 \times 10^{-13}$	connected + disconnected
scalar loop, $\vec{p} = (1,0,0)$	NumPy contraction	64	3.4×10^{-15}	64 time slices
$\Delta^+ = \Delta^{++}$, $\Lambda = p$	exact identity	64	$\leq 6.1 \times 10^{-16}$ (rel.)	2 configurations
NN: $\frac{1}{\sqrt{2}}(np + pn) = pp$, $I = 0$ $^1S_0 = 0$	exact identity	64	$\leq 5.6 \times 10^{-16}$ (rel.)	2 configurations

both the DeGrand–Rossi and Dirac–Pauli bases, the source adjoint of derivative and transported fields, and the explicit rejection of unsupported requests such as temporal derivatives. In the absence of cancellations the number of Wick terms of a correlator equals the number of flavor-allowed pairings, $\prod_f n_f!$, where n_f is the number of quark fields of flavor f; the counts of Table 1 satisfy this relation. A maintained numerical oracle feeds random color-ϵ elementals and distinct up- and down-quark propagation tensors through the production runtime and compares the complete local-proton spin matrix with an explicit direct-minus-exchange Einstein contraction; it also verifies the DP positive-parity trace and the covariance between the DR and DP bases. Because this oracle does not depend on any file layout, it protects the baryon Wick, spin and runtime-axis algebra between real-data validation campaigns.

7.3. Numerical correlator validation

Table 2 lists the real-data comparisons. Mesons, bilocal two-meson operators and the derivative and gauge-transported operators are compared with Chroma or Chroma-LapH; baryon-meson systems, the Δ^{++} and the disconnected loops with independent NumPy contractions; and the multi-baryon systems, for which no external reference was available to us, with exact flavor identities.

Several entries deserve comment. The derivative and transported pion channels fix the derivative normalization $D_i = \frac{1}{2}(T_{+i} - T_{-i})$, the path ordering of $T_{+x,+y}$ and the source adjoint; they were compared with the original Chroma-LapH elementals at four source–sink separations. The local four-quark operator agrees with Chroma once the different normalization convention is taken into account: SymCorr evaluates every operator as the unnormalized sum $\sum_{\vec{x}} e^{i\vec{p}\cdot\vec{x}} O(\vec{x})$, whereas the external code includes one extra factor of the spatial volume per insertion for this operator; no global correction is applied. The baryon-meson references include the terms with equal-time ("same time slice") propagation separately, because these dominate the raw correlators and would otherwise mask errors in the cross-time terms. The $N\sigma$ and $\bar{u}u$ results are not vacuum subtracted; vacuum subtraction is part of the ensemble analysis, not of the contraction.

For baryons beyond the single nucleon and the Δ, exact identities take the place of an external reference. With $D(q_1, q_2) = \epsilon_{abc}(q_{1a}^T C\gamma_5 q_{2b})$ and $D_i(q_1, q_2) = \epsilon_{abc}(q_{1a}^T C\gamma_i q_{2b})$,

$$\begin{aligned} \Delta_i^+ &= \frac{1}{\sqrt{3}}\left[2D_i(u,d)\,u + D_i(u,u)\,d\right], \\ \Lambda &= \frac{1}{\sqrt{6}}\left[2D(u,d)\,s + D(u,s)\,d - D(d,s)\,u\right], \\ \langle \Delta_i^+ \bar{\Delta}_j^+ \rangle &= \langle \Delta_i^{++} \bar{\Delta}_j^{++} \rangle, \qquad \langle \Lambda\bar{\Lambda} \rangle = \langle p\,\bar{p} \rangle, \end{aligned} \tag{29}$$

hold exactly when u, d and s are bound to the same perambulator, although the compared correlators contain different numbers of Wick terms (8 versus 6 and 9 versus 2). For two nucleons, each projected separately to zero momentum and coupled with $C\gamma_5$, the $I = 1$, $I_3 = 0$ combination $(n_x p_y + p_x n_y)/\sqrt{2}$ (144 Wick terms) must reproduce pp (48 terms), and the $I = 0$ spin-singlet combination $(n_x p_y - p_x n_y)/\sqrt{2}$ must vanish because both nucleons occupy the same spatial state. Changing one coefficient in the Δ^+ or Λ operator changes the result by 20–50%, and the forbidden two-nucleon channel is below 2.1×10^{-16} of the pp correlator.

7.4. End-to-end and regression testing

The production path is exercised end to end by the regression suite: a job is planned, the portable plan is saved and reloaded, synthetic HDF5 eigenvector and perambulator files are bound, and the persisted output is compared with independent formulas. Further tests check that batched and unbatched execution agree, that host-staged tiling reproduces resident execution, that propagation prefetch does not change results, that interrupted jobs resume only the unfinished work units, and that a stale plan or a result file from a different run is rejected. The CUDA-specific variants of these tests run only where a device is present.

On real data, the complete 64×64 source–sink grid of a rest-frame pion and a positive-parity nucleon (4096 work units each, $N_v = 64$) was computed from the same saved plans on a 64-core CPU node and on one NVIDIA H100. The two backends agree to 5.4×10^{-15} (pion) and 1.7×10^{-15} (nucleon) relative to the channel magnitude. Validation in this section is bounded by what was compared: three-point functions, generalized (POINT-current) propagation, disconnected loops beyond the local scalar and γ_4 cases and the $\vec{p} = (1, 0, 0)$ scalar loop, derivative operators with other spin structures or displaced baryon legs, and multi-device execution are implemented or planned but are not covered by Table 2.

8. Worked Examples

The operator examples of Sec. 3 showed how single hadrons are written. This section follows complete calculations from the operator file to the stored correlators, using systems that are tedious to set up by hand: a nucleon–pion correlation matrix, a two-nucleon system with an exact isospin check, and the triton.

8.1. A nucleon–pion correlation matrix

The $I = \frac{1}{2}$, $I_3 = +\frac{1}{2}$ nucleon–pion basis of Table 3 combines a local proton, six proton operators with a gauge-transported third quark and the two-hadron operators

$$(N\pi)_{\vec{p}} = \sqrt{2/3}\; n(\vec{p})\, \pi^+(-\vec{p}) - \sqrt{1/3}\; p(\vec{p})\, \pi^0(-\vec{p}). \quad (30)$$

Using small helper functions for the hadrons (with the conventions of Sec. 3), the operator file and the job file are shown in Fig. 5. The helpers `pi_plus` and `pi_zero` return $\bar{d}\gamma_5 u$ and $(\bar{u}\gamma_5 u - \bar{d}\gamma_5 d)/\sqrt{2}$ at the given position and momentum, and the open nucleon spin index `sN` becomes a 4×4 output axis of every correlator. Index labels only need to be distinct within one operator (the pion helpers use their own labels); every insertion of an operator into a correlator is renamed into a separate namespace. In the job file, `{configuration}` and `{source_time}` are expanded per work unit.

The gauge links are needed only because of the transported quark. Because π^0 contains $\bar{u}u$ and $\bar{d}d$ at one point, some Wick terms contain propagation from a time slice to itself, so the perambulators are read in the all-time layout. For the first 13 operators (169 correlators), `symcorr plan` followed by `symcorr inspect` reports

```
Contraction:     reuse-aware
Selected paths:  greedy
Resources:       65 (58 kernel, 5 spin, 2 propagation)
Execution:       18115 physical steps, 4842 buffers,
                 peak active 1.45 GiB
Wick terms       5594/5594 surviving
```

(a) `npi_ops.py`

```
def nucleon(q3, pos, p=(0, 0, 0), hop=None):
    f = q3(pos, "c", "sN")
    if hop is not None:
        f = transport(f, hop)
    return Operator(
        epsilon("a", "b", "c")
        * spin_matrix("Cg5", "al", "be")
        * u(pos, "a", "al") * d(pos, "b", "be")
        * f, momentum=p)

def build(max_shell=2):
    ops = [nucleon(u, "x")]
    ops += [nucleon(u, "x", hop=h) for h in
            ("+x", "-x", "+y", "-y", "+z", "-z")]
    for p in momenta(max_shell):  # 19 momenta
        mp = neg(p)
        ops.append(
            sqrt(2/3) * nucleon(d, "x", p)
                      * pi_plus("y", mp)
          - sqrt(1/3) * nucleon(u, "x", p)
                      * pi_zero("y", mp))
    return {f"C_{i}_{j}": c
            for i, j, c in CorrMatrix(ops)}
```

(b) `npi_job.yaml`

```
operators: {file: npi_ops.py, factory: build}
lattice: {spatial_extent: [24, 24, 24],
          temporal_extent: 64}
compile:
  mode_size: 32
  scalar_type: complex128
  spin_basis: DP
  contraction: {strategy: reuse-aware}
run:
  device: auto
  configurations: {values: [1]}
  times:
    generator: {type: two_point, source: 0,
                sinks: {range: {start: 1, stop: 9}}}
data:
  eigenvectors:
    provider: hdf5_distillation
    path: laphev_{configuration}.h5
    dataset: /eigenvectors
    axes: [time, z, y, x, color, mode]
  gauge_links: {provider: laph_iomap,
                path: gauge_{configuration}}
  propagation:
    provider: hdf5_perambulator
    ordinary:
      files: {u: peram_{configuration}.h5,
              d: peram_{configuration}.h5}
      dataset: /perambulators/src_t_{source_time}/data
      source_field: qdagger
      axes: [sink_time, source_spin, sink_spin,
             source_mode, sink_mode]
```

Figure 5: Complete input of the nucleon–pion example of Sec. 8.1: (a) the operator factory, (b) the job file. `momenta`, `neg`, `pi_plus` and `pi_zero` are short helpers in the same file.

```
components       4308/8162 unique
intermediates    12273/62108 unique
est. FLOPs       1.07e12 scheduled;
                 7.15e11 saved by intermediate CSE
```

without reading any lattice data. The 5594 Wick terms of the 169 correlators contain 8162 connected networks, of which 4308 are distinct; the pairwise contractions reduce from 62 108 to 12 273. `Selected paths: greedy` shows the fallback of Sec. 6.3: the $N\pi$ networks have more than twelve tensors. The run binds the 58 kernels and two perambulators for each time slice and writes one dataset of shape $(1, 8, 4, 4)$ per correlator, together with the completion mask.

8.2. Two nucleons and an exact isospin check

Two-baryon operators are products of single-baryon operators with a spin coupling, here $C\gamma_5$ between the open nucleon indices `s1` and `s2`:

```
C = spin_matrix("Cg5", "s1", "s2")
pp  = proton("x", open_spin="s1") \
      * proton("y", open_spin="s2") * C
nn0 = (neutron("x", open_spin="s1")
         * proton("y", open_spin="s2")
       + proton("x", open_spin="s1")
         * neutron("y", open_spin="s2")) * C / sqrt(2)
```

The pp correlator has 48 Wick terms and the $I_3 = 0$ member of the same isospin multiplet has 144. When the up and down quarks are bound to the same perambulator file, the two correlators must agree configuration by configuration, and the antisymmetric combination with $-$ instead of $+$ must vanish (Sec. 7.3). At $N_v = 64$ both relations hold to 6×10^{-16} relative to the pp correlator on two configurations, which checks the sign of every one of the 192 Wick terms of the two-nucleon system. The same operators with back-to-back momenta give the 17-operator NN bases of Sec. 9.

8.3. The triton

Three-nucleon operators are written in the same way,

```
triton = proton("x", open_spin="sN") \
         * neutron("y", open_spin="s1") \
         * neutron("z", open_spin="s2") \
         * spin_matrix("Cg5", "s1", "s2")
```

and the single correlator has $5!\,4! = 2880$ Wick terms, all of which are connected nine-quark networks. The difficulty is not the Wick contraction but the contraction order. With greedy paths the plan for one work unit at $N_v = 32$ needs 2.6×10^{15} FLOPs and a 2.9 TB buffer pool; restricting the size of every intermediate to 10^9 elements with `contraction.memory_limit` reduces this to 2.3×10^{14} FLOPs and 117 GB, and the `auto-hq` search to 2.8×10^{13} FLOPs and 31 GB, which runs on a single CPU node (Sec. 9). These numbers are available from `symcorr inspect` before any data are touched, so a user can choose the strategy before committing compute time.

9. Performance Evaluation

The cost of a distillation calculation is dominated by the contractions, and the practical question for a user is how this cost grows when the operator basis grows, how much of it SymCorr removes automatically, and what is needed to run it. We answer these questions for five two-hadron systems and two light nuclei that contain the contraction topologies of Sec. 4.3. All numbers in this section are measured or taken from the analytical cost report of the compiled plan; none is extrapolated.

9.1. Benchmark setup

Lattice data We use one gauge configuration of a locally generated $N_f = 3$ ensemble with the parameters of the CLS ensemble A653 ($\beta = 3.34$, $\kappa_u = \kappa_d = \kappa_s = 0.1365716$) but a larger time extent, $24^3 \times 64$. The LapH subspace is spanned by up to 64 eigenvectors of the gauge-covariant three-dimensional Laplacian built from stout-smeared links, and the light-quark perambulators are available for all 64 source times. For the doubly-bottom system we bind the b quark to a heavy-quark perambulator generated on the same configuration for a separate bottomonium study. Flavor assignments affect only the Wick structure and the data that are read, so the timings do not depend on the physics of these choices.

Operator bases Table 3 lists the bases. Multi-hadron operators are products of separately momentum-projected hadrons at distinct positions with back-to-back momenta $\pm\vec{p}$ (in units of $2\pi/L$); they are not projected onto lattice irreducible representations, because a projected operator is a fixed linear combination of these and does not change the contraction work. For two identical bosons, and for two nucleons coupled with a symmetric or antisymmetric spin matrix, the operators with $\vec{p}$ and $-\vec{p}$ coincide, so these bases contain one momentum per pair. The $N\pi$ basis also contains the proton with its third quark transported by one link in each of the six spatial directions (Fig. 2), so gauge links are read and transported kernels are built. Every basis of N_{op} operators defines the $N_{\mathrm{op}} \times N_{\mathrm{op}}$ correlation matrix, and smaller bases are the leading N_{op} operators of the list.

Hardware and measurement Plans were compiled single-threaded on an Intel Xeon Gold 6438N. Numerical runs used (i) one CPU node with two 32-core Intel Xeon Platinum 8358 processors and 64 PyTorch threads, and (ii) a workstation GPU, an NVIDIA RTX PRO 2000 Blackwell with 16 GB of memory and low double-precision throughput, hosted by an Intel Core i9-14900K. Software versions were Python 3.13 and PyTorch 2.13 (CPU) and Python 3.12 and PyTorch 2.11 with CUDA 12.8 (GPU), with `opt_einsum` 3.4 and h5py 3.16. All runs use complex128 arithmetic, the DP spin basis, source time $t_0 = 0$ and the eight sink times $t = 1, \ldots, 8$, i.e. eight work units. Times are taken from the run record of `symcorr run` (Sec. 6.6); the plan was compiled beforehand and compile times are reported separately. We quote the numerical-execution time per work unit and, where useful, the throughput defined as the scheduled FLOPs of the plan divided by that time.

Table 3
Benchmark systems. N_{op} is the size of the full basis; "Wick terms" counts the terms of the full correlation matrix after exact cancellation. $\pi^0 = (\bar{u}\gamma_5 u - \bar{d}\gamma_5 d)/\sqrt{2}$, $\sigma = (\bar{u}u + \bar{d}d)/\sqrt{2}$, $N_{C\gamma_5} = \epsilon_{abc}(u_a^T C\gamma_5 d_b)q_c$.

System	Operators	N_{op}	Wick terms	Topologies
$N\pi$, $I = \frac{1}{2}$	p; p with transported third quark (6); $\sqrt{2/3}\,n(\vec{p})\pi^+(-\vec{p}) - \sqrt{1/3}\,p(\vec{p})\pi^0(-\vec{p})$, $\lvert\vec{p}\rvert^2 \le 2$ (19)	26	47 142	exchange, equal-time
$\pi\pi$, $I = 0$	σ; $(\pi^+\pi^- + \pi^-\pi^+ - \pi^0\pi^0)/\sqrt{3}$ at $\pm\vec{p}$, $\lvert\vec{p}\rvert^2 \le 4$ (17)	18	60 410	loops, vacuum
$bb\bar{u}\bar{d}$, $J^P = 1^+$	$\epsilon\epsilon\,(b^T C\gamma_3 b)(\bar{u}C\gamma_5\bar{d}^T)$; $BB^*(\pm\vec{p})$ and $B^*B^*(\pm\vec{p})$, $\lvert\vec{p}\rvert^2 \le 2$ (19 each)	39	11 858	local four-quark
NN, $^1S_0$, $I = 1$	$p(\vec{p})\,C\gamma_5\,p(-\vec{p})$, $\lvert\vec{p}\rvert^2 \le 4$	17	13 872	six-quark exchange
NN, $^3S_1$, $I = 0$	$[p(\vec{p})\,C\gamma_3\,n(-\vec{p}) - n(\vec{p})\,C\gamma_3\,p(-\vec{p})]/\sqrt{2}$, $\lvert\vec{p}\rvert^2 \le 4$	17	41 616	six-quark exchange
^{3}H	$p(x)\,[n(y)\,C\gamma_5\,n(z)]$, $\vec{p} = 0$	1	2 880	nine-quark exchange
^{4}He	$[p(x)\,C\gamma_5\,p(y)]\,[n(z)\,C\gamma_5\,n(w)]$, $\vec{p} = 0$	1	518 400	twelve-quark exchange

9.2. Reuse across a correlation matrix

Figure 6 shows how the work grows with the number of correlators N_{op}^2 at $N_v = 64$, and Table 4 lists the full bases. The dashed lines describe evaluating every Wick term as an independent tensor network with the same contraction paths; the solid lines describe the compiled plan. Three effects are visible.

First, the number of pairwise contraction steps grows much more slowly than the number of term-level steps: for the full bases, 67–82% of all steps are duplicates that are computed once. The removed steps are mostly the cheap ones (contractions of spin matrices with perambulators, meson blocks, partial baryon blocks), so the FLOP reduction is smaller, a factor 1.7–2.8. The most expensive steps of different networks are generally different, because the contraction path of each network is chosen separately (Sec. 11).

Second, the memory reduction is large and grows with the basis. Storing every intermediate value of the full $N\pi$ matrix would require 6.1 TB per work unit; the scheduled plan with lifetime-based buffer reuse needs 35 GB at the same N_v, a factor 170. For the two-nucleon matrices the factors are 46 ($^1S_0$) and 101 ($^3S_1$). Unlike the FLOP count, this reduction does not depend on the cleverness of the contraction path but only on the schedule.

Third, for systems whose cost is dominated by one expensive operator, adding cheap operators is almost free. In the $bb\bar{u}\bar{d}$ basis the local diquark–antidiquark operator has a four-index LapH kernel, and its correlator alone needs 2.8×10^{11} FLOPs at $N_v = 64$ (growing as N_v^6 with greedy paths), while a meson–meson correlator grows as N_v^3; the 1521 correlators of the full basis need only twice the FLOPs of the single local correlator.

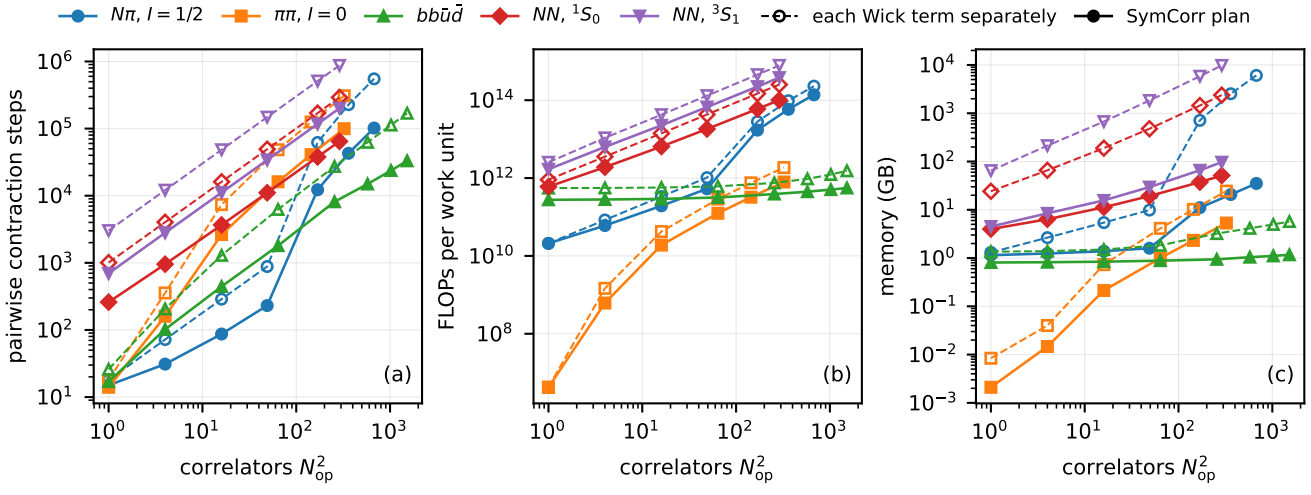


Figure 6: Compile-time cost report versus the number of correlators N_{op}^2 at $N_v = 64$, greedy contraction paths. (a) Pairwise contraction steps, (b) contraction FLOPs per work unit, (c) memory. Dashed: every Wick term evaluated as an independent network and every intermediate stored. Solid: the compiled SymCorr plan after global and partial-intermediate reuse, scheduling and buffer reuse (physical peak).

Table 4
Full correlation matrices at $N_v = 64$, greedy contraction paths. Networks and steps are given as unique/occurrences. FLOPs and memory per work unit are given before → after reuse, with the reduction factor in parentheses. Compile time on one core.

System	N_{op}	corr.	Wick terms	networks	steps	FLOPs	memory (GB)	compile (min)
$N\pi$, $I = 1/2$	26	676	47 142	36457/70588	101869/550973	$2.29\times10^{14} \to 1.38\times10^{14}$ (1.7)	6085 → 35.1 (173)	85
$\pi\pi$, $I = 0$	18	324	60 410	25716/142062	99672/308260	$1.86\times10^{12} \to 7.92\times10^{11}$ (2.4)	24 → 5.3 (5)	132
$bb\bar{u}\bar{d}$	39	1521	11 858	17634/17634	33095/169322	$1.54\times10^{12} \to 5.56\times10^{11}$ (2.8)	6 → 1.2 (5)	4
NN, $^1S_0$	17	289	13 872	13872/13872	64576/291312	$2.52\times10^{14} \to 1.01\times10^{14}$ (2.5)	2379 → 51.3 (46)	6
NN, $^3S_1$	17	289	41 616	41616/41616	200794/873936	$7.84\times10^{14} \to 3.81\times10^{14}$ (2.1)	9809 → 96.9 (101)	54

Table 5
Contraction-path strategies at fixed basis size: scheduled FLOPs per work unit / physical peak memory (GB) / compile time (s). The `reuse-aware` entry equals one of the other columns by construction.

System	N_{op}	N_v	`greedy`	`dynamic-programming`	`auto-hq`	`reuse-aware`
$N\pi$	1	64	2.09×10^{10} / 1.15 / 0	4.33×10^{9} / 0.145 / 0	4.33×10^{9} / 0.145 / 0	4.33×10^{9} / 0.145 / 0
$N\pi$	7	64	5.30×10^{11} / 1.6 / 1	5.43×10^{10} / 0.197 / 1	5.43×10^{10} / 0.197 / 1	5.43×10^{10} / 0.197 / 2
$\pi\pi$ $(I = 0)$	8	64	1.25×10^{11} / 0.999 / 172	1.65×10^{11} / 0.28 / 283	1.65×10^{11} / 0.28 / 310	1.25×10^{11} / 0.999 / 298
$bb\bar{u}\bar{d}$	8	64	3.16×10^{11} / 0.883 / 4	2.56×10^{11} / 4.86 / 10	2.56×10^{11} / 4.86 / 10	3.16×10^{11} / 0.883 / 6
NN $(^1S_0)$	4	64	6.46×10^{12} / 11.3 / 9	4.59×10^{12} / 4.32 / 404	4.24×10^{12} / 4.28 / 94	6.46×10^{12} / 11.3 / 14
NN $(^3S_1)$	2	64	6.31×10^{12} / 8.45 / 7	2.92×10^{12} / 3.23 / 223	2.53×10^{12} / 3.56 / 70	6.31×10^{12} / 8.45 / 11
^{3}H	1	16	2.05×10^{13} / 117 / 74	–	4.70×10^{11} / 2.25 / 611	–
^{3}H	1	32	2.65×10^{15} / 2.86e+03 / 74	–	2.78×10^{13} / 30.5 / 602	2.65×10^{15} / 2.86e+03 / 119
^{3}H	1	48	–	–	2.66×10^{14} / 127 / 612	–
^{3}H	1	64	1.67×10^{17} / 8.02e+04 / 75	–	1.49×10^{15} / 407 / 604	–

9.3. Effect of the contraction path

Table 5 compares the path strategies of Sec. 6.3 on one representative basis per system. The differences are larger than those from reuse. For a single proton, the greedy path forms a larger intermediate; it needs 4.8 times more FLOPs than the exact search at every N_v (both scale as N_v^4) and 8 times more memory at $N_v = 64$; for the $N\pi$ matrix with seven operators the factors are 9.8 and 8. For the two-nucleon matrices `auto-hq` reduces FLOPs by 1.5–2.5 and memory by 2.4–2.6. For the triton the effect decides feasibility: at $N_v = 32$ the greedy plan needs 2.6×10^{15} FLOPs and a 2.9 TB buffer pool per work unit, whereas the `auto-hq` plan needs 2.8×10^{13} FLOPs and 31 GB. Path choice and reuse also interact. For $\pi\pi$ the per-network optimal paths of dynamic programming are cheaper before reuse (2.83×10^{11} versus 2.88×10^{11} FLOPs) but share fewer intermediates, so after reuse greedy is cheaper (1.25×10^{11} versus 1.65×10^{11}); `reuse-aware` detects this because it compares complete plans after reuse. For $bb\bar{u}\bar{d}$ dynamic programming saves 19% of the FLOPs but needs 5.5 times more memory, and `reuse-aware` keeps the greedy plan because of its memory ceiling. The exact searches are more expensive to compile and are limited to small networks; `reuse-aware` therefore falls back to greedy paths for the multi-baryon systems, where `auto-hq` has to be requested explicitly.

10. Software Availability and Reproducibility

10.1. Package and dependencies

SymCorr 0.1.0 is a pure Python package (Python ≥ 3.11) available at `https://github.com/Tanishk-Shrimal/Symcorr`. Its run-time dependencies are NumPy, PyTorch (CPU or CUDA builds), h5py, `opt_einsum`, Pydantic, PyYAML and Typer; no compiled extension, MPI library or vendor-specific GPU code is required. The same package therefore runs on a laptop, a CPU cluster node or a CUDA GPU, and the device is selected at run time. The repository contains the regression suite (1524 tests), the validation tools of Sec. 7, and documentation of the physics conventions (spin bases, phase and normalization conventions, storage layouts).

10.2. User workflow

A calculation is specified by two files: a Python file that builds the operators and correlators with the symbolic interface of Sec. 3, and a YAML job file that names the operator factory and its parameters, the lattice extents, the compile options (N_v, scalar type, spin basis, contraction strategy), the run options (configurations, source and sink times, device, time batch size) and the data sources (eigenvectors, smeared gauge links, perambulators and their storage layout). The two stages are separate commands:

```
symcorr plan job.yaml      # compile -> plan.symcorr
symcorr inspect plan.symcorr --hardware
symcorr run job.yaml       # execute -> results.h5
```

`symcorr plan` needs no lattice data. The portable plan stores the compiled execution state together with a fingerprint of the operator source (including its local Python helpers), the factory parameters and the compile options; data paths, configurations, time contexts and the device are excluded, so one plan can be compiled once and run on many configurations and machines. `symcorr inspect` prints the analytical cost report of Sec. 9 (Wick terms, unique networks and intermediates, FLOPs, memory) and, with `--hardware`, checks the plan against the current device. `symcorr`

`run` rejects a plan whose fingerprint no longer matches the job and resumes an interrupted output file.

10.3. Reproducibility of the results in this work

All results in this paper were obtained with the SYMCORR revision tagged `v0.1.0`. The operator definitions, job generators, timing and compile-scan drivers and the plotting scripts used for Sec. 9 are provided in the `benchmarks/paper` directory of the repository, together with the machine-readable run records. The validation tools and reports of Sec. 7 are in `tools/` and `docs/validation.md`. The lattice data used here (eigenvectors, perambulators and smeared gauge fields of one configuration) are available from the authors on request.

11. Limitations and Future Development

We separate what is validated, what is implemented but not yet validated with real data, and what is planned.

Validated scope The validated scope of version 0.1.0 is the content of Table 2: single- and two-hadron two-point functions built from local, derivative and gauge-transported quark fields in exact distillation, baryon and multi-baryon correlators (through independent contractions and exact identities), and local closed loops at zero and one unit of momentum.

Implemented, not yet validated Three-point functions with current insertions, generalized (POINT-current) propagation resources, disconnected loops with other spin structures, momenta or displacements, and displaced baryon legs other than the tested spectator transport are implemented and covered by synthetic tests, but have not been compared with an independent real-data reference. Temporal derivatives are rejected explicitly.

Algorithmic limitations Contraction paths are chosen per connected network. Cross-correlator reuse is therefore found only for intermediates that happen to coincide in the chosen trees; Sec. 9 shows that this removes two thirds to four fifths of the pairwise contraction steps but only 40–65% of the FLOPs, because the most expensive steps of different networks are usually not shared. A path search that targets shared hadron blocks across the whole correlation matrix is the most promising extension. The exact path searches of the `reuse-aware` strategy are restricted to networks with at most twelve tensors, so multi-baryon calculations currently fall back to greedy paths unless `auto-hq` is requested explicitly. Equalities that rely on tensor symmetries or on Hermiticity (for example C_{12} and C_{21}, Sec. 6.2.4) are not exploited. Operators whose quarks are all at one point, such as a local six-quark operator, lead to rank-six LapH kernels and are impractical in the present exact distillation path.

Execution Execution uses one process and one device. Memory not modeled by the plan (provider workspaces, library temporaries) is covered by a fixed safety reserve, which can be insufficient on small GPUs (Sec. 9). Multi-GPU and distributed execution, out-of-core host storage and CPU-thread tuning are not implemented.

Future developments Natural extensions that fit the present architecture are: stochastic distillation and other sampled LapH-noise estimators, which replace the exact perambulator by noise-diluted propagation resources; stochastic estimation of disconnected contributions with dedicated variance-reduction techniques; optimized distillation profiles, in which the LapH projector is replaced by a weighted or operator-dependent combination of eigenvectors; reuse-driven contraction-path selection across the correlation matrix; and multi-device execution of independent work units.

12. Conclusions

References

[1] William Detmold, William I. Jay, Gurtej Kanwar, Phiala E. Shanahan, and Michael L. Wagman. Multiparticle interpolating operators in quantum field theories with cubic symmetry. *Phys. Rev. D*, 109:094516, May 2024.

[2] Haobo Yan, Chuan Liu, Liuming Liu, and Yu Meng. Construction of general N-body lattice operators with arbitrary momenta. *JHEP*, 10:210, 2025.

[3] Dalibor Djukanovic. Quark Contraction Tool — QCT. *Comput. Phys. Commun.*, 247:106950, 2020.

[4] R. G. Edwards, B. Joó, et al. The Chroma software system for lattice QCD. In *Lattice field theory. Proceedings, 22nd International Symposium, Lattice2004, Batavia, USA, June 21-26, 2004*, volume 140, pages 832–835, 2005.

[5] Jie Chen, Robert G. Edwards, and Weizhen Mao. Graph contractions for calculating correlation functions in lattice qcd. In *Proceedings of the Platform for Advanced Scientific Computing Conference*, PASC '23, New York, NY, USA, 2023. Association for Computing Machinery.

[6] Michael Peardon, John Bulava, Justin Foley, Colin Morningstar, Jozef Dudek, Robert G. Edwards, Balint Joo, Huey-Wen Lin, David G. Richards, and Keisuke Jimmy Juge. A Novel quark-field creation operator construction for hadronic physics in lattice QCD. *Phys. Rev. D*, 80:054506, 2009.

[7] Debsubhra Chakraborty, Piyush Srivastava, Arpith Kumar, and Nilmani Mathur. Nuclear correlation functions using first-principle calculations of lattice quantum chromodynamics. *Phys. Rev. D*, 110:114505, Dec 2024.